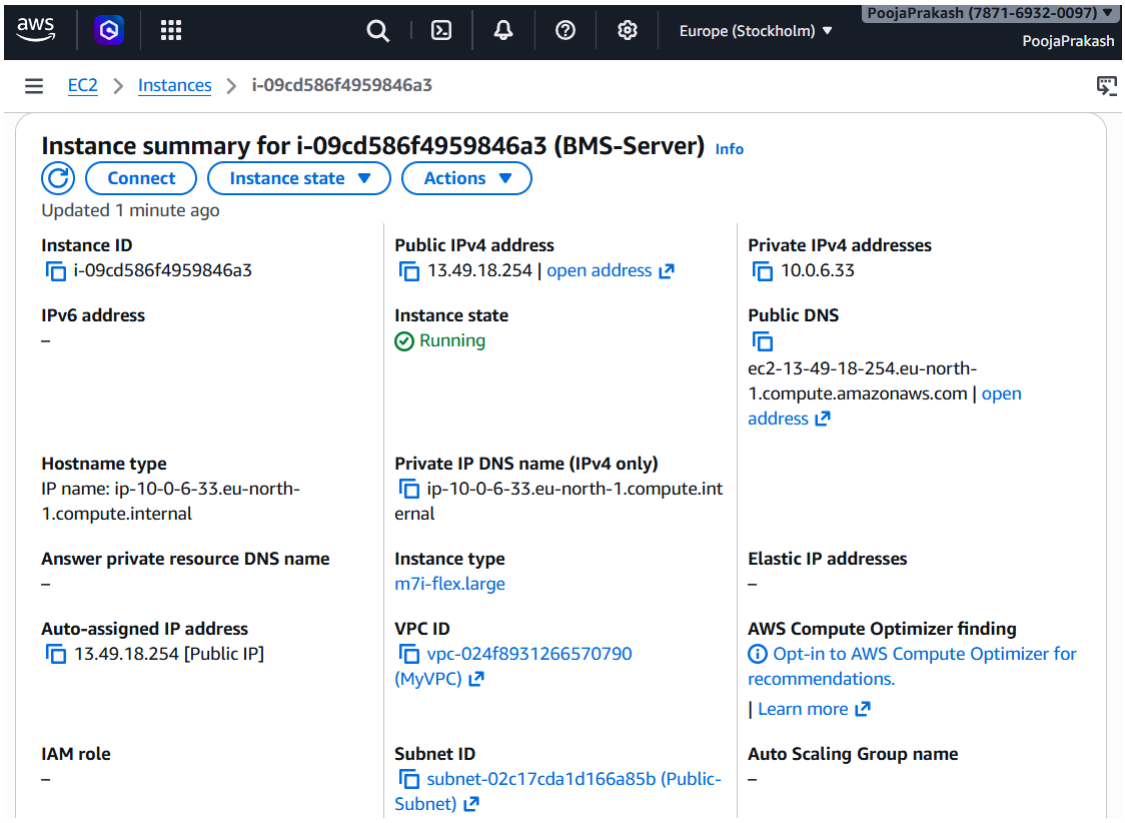
CI/CD Implementation and Kubernetes Deployment of Book My Show Application

Name: Pooja Prakash Belgaumkar

This project demonstrates the end-to-end deployment and monitoring of the Book My Show application using modern DevOps practices. The objective of this project is to implement Continuous Integration, Continuous Deployment, containerization, security scanning, monitoring and Kubernetes orchestration using industry-standard tools such as Jenkins, Docker, SonarQube, OWASP, Trivy, Prometheus, Grafana and Amazon EKS.

Phase 1: CI/CD Setup on BMS Server

- Launched an Ubuntu EC2 instance named BMS-Server and configured necessary inbound rules.



Inbound rules (8)

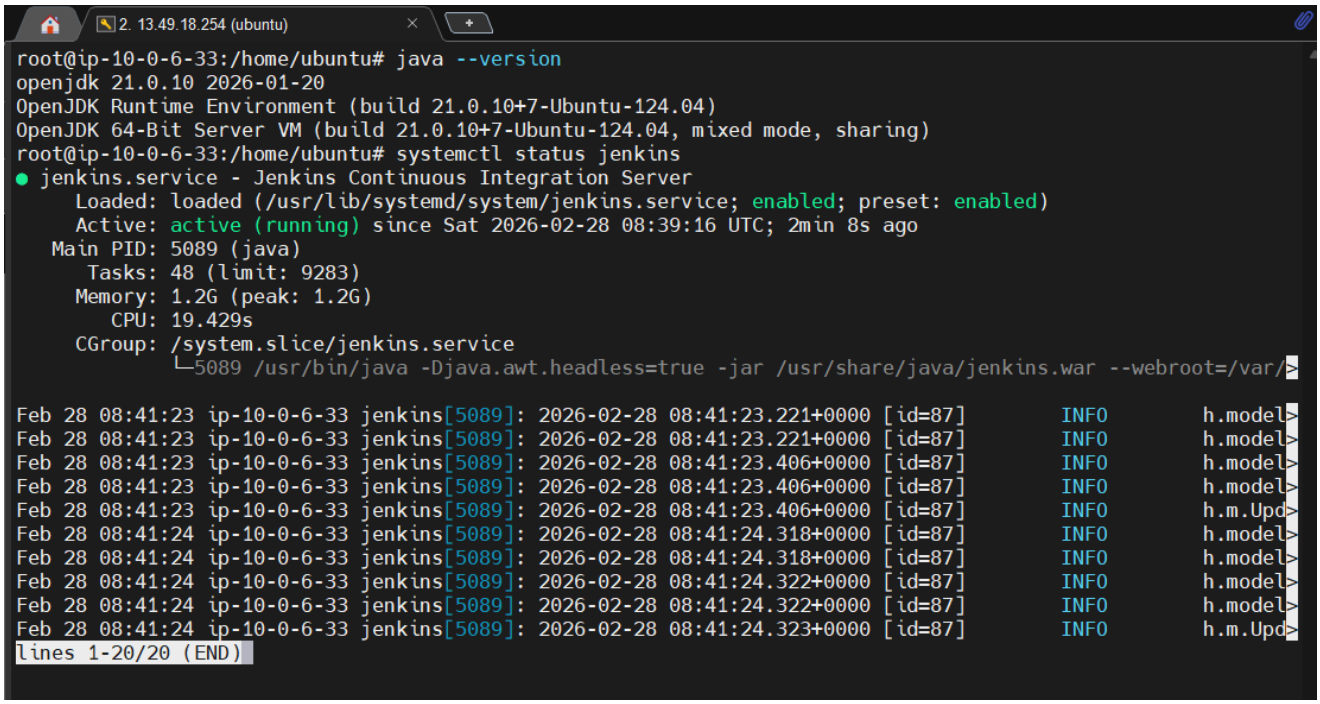
Manage tags | Edit inbound rules

Search

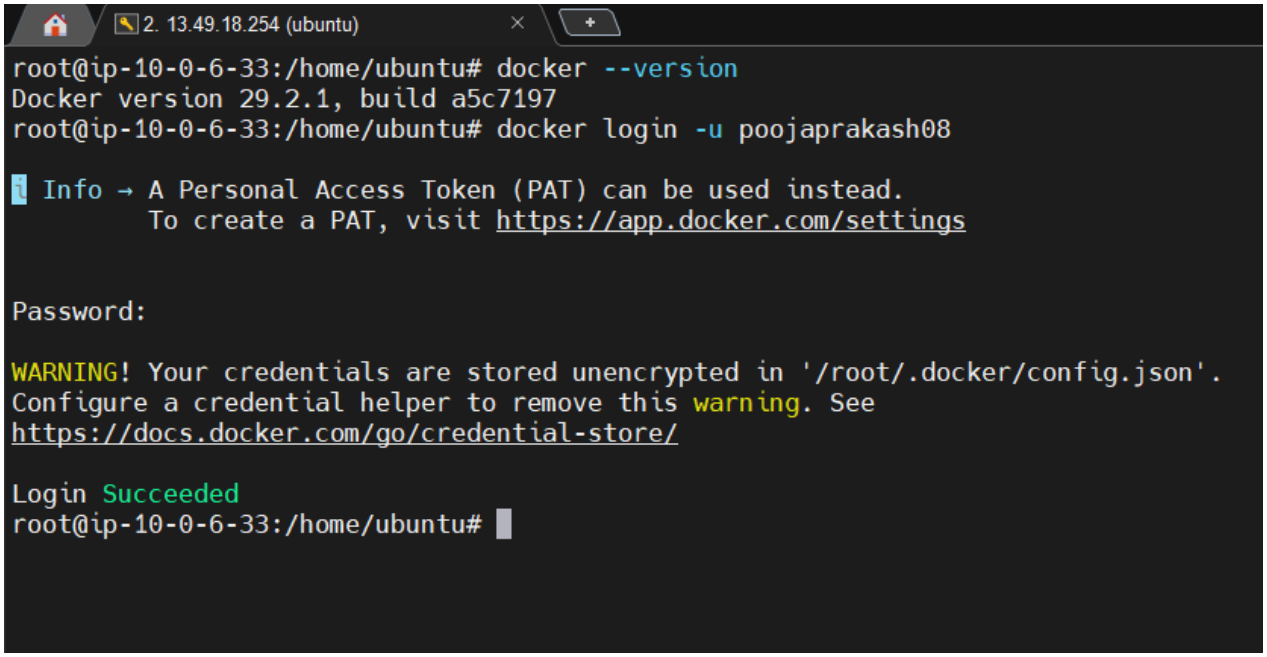
1

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
–	sgr-0d160482a5cd1d125	IPv4	SMTPS	TCP	465	0.0.0.0/0	Email Communication
–	sgr-0b1713ed51333c97d	IPv4	HTTPS	TCP	443	0.0.0.0/0	–
–	sgr-07f169caf5294386b	IPv4	SSH	TCP	22	0.0.0.0/0	–
–	sgr-0f0aaef6a1951089e	IPv4	Custom TCP	TCP	30000 - 32767	0.0.0.0/0	Nodeport Services
–	sgr-043f41d596d4d317e	IPv4	SMTP	TCP	25	0.0.0.0/0	Email Communication
–	sgr-0ad015c03e7a79c3b	IPv4	HTTP	TCP	80	0.0.0.0/0	–
–	sgr-0b3e0d6bb16d32ecf	IPv4	Custom TCP	TCP	6443	0.0.0.0/0	–
–	sgr-08e515c7454082645	IPv4	Custom TCP	TCP	3000 - 10000	0.0.0.0/0	Grafana-Jenkins-SonarQube-Prometheus

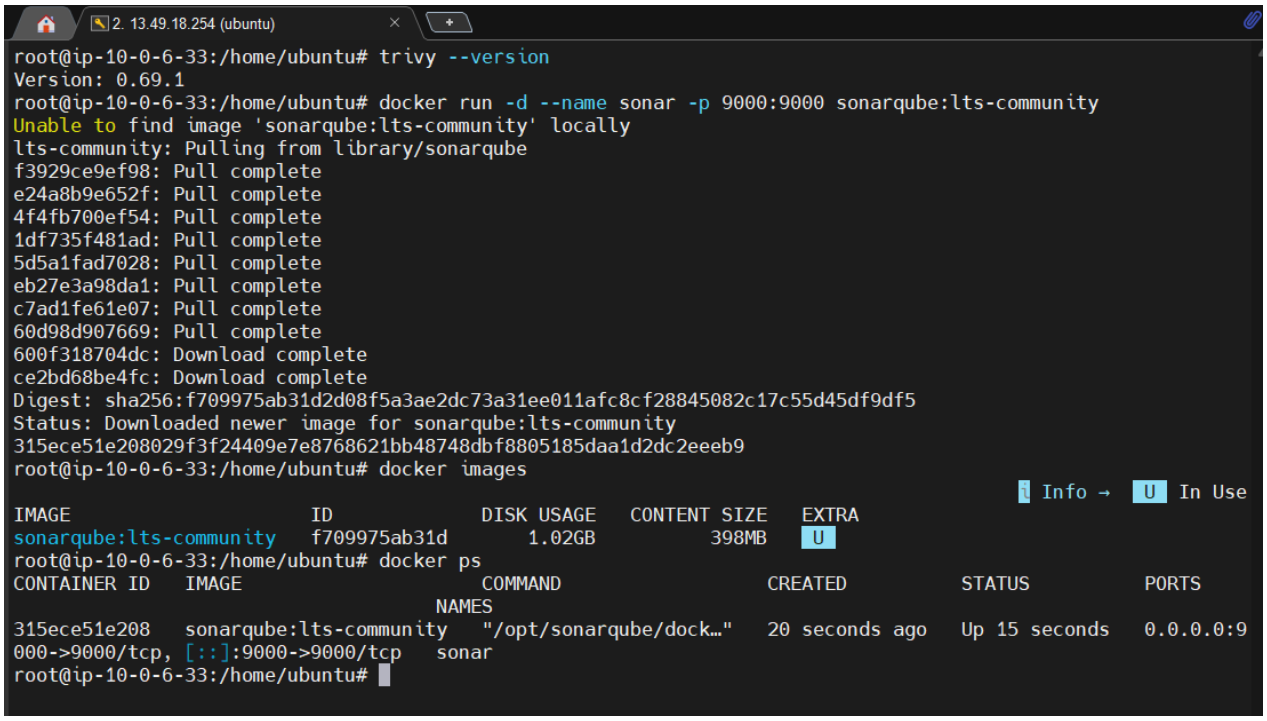
- Connected to the instance using MobaXterm and installed Java and Jenkins.



- Installed Docker and logged in to Docker Hub using valid credentials to push Docker images.

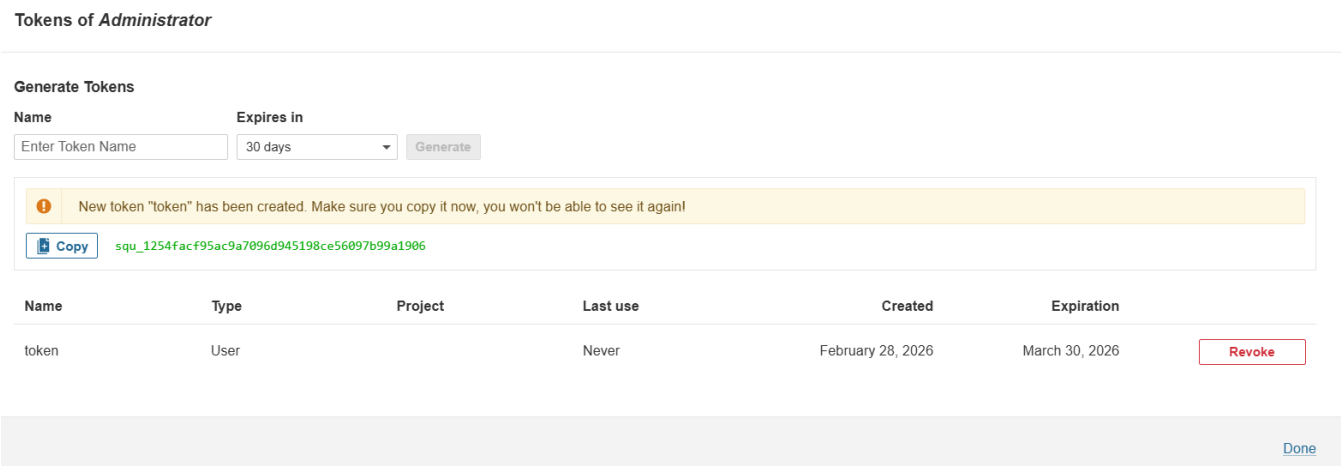


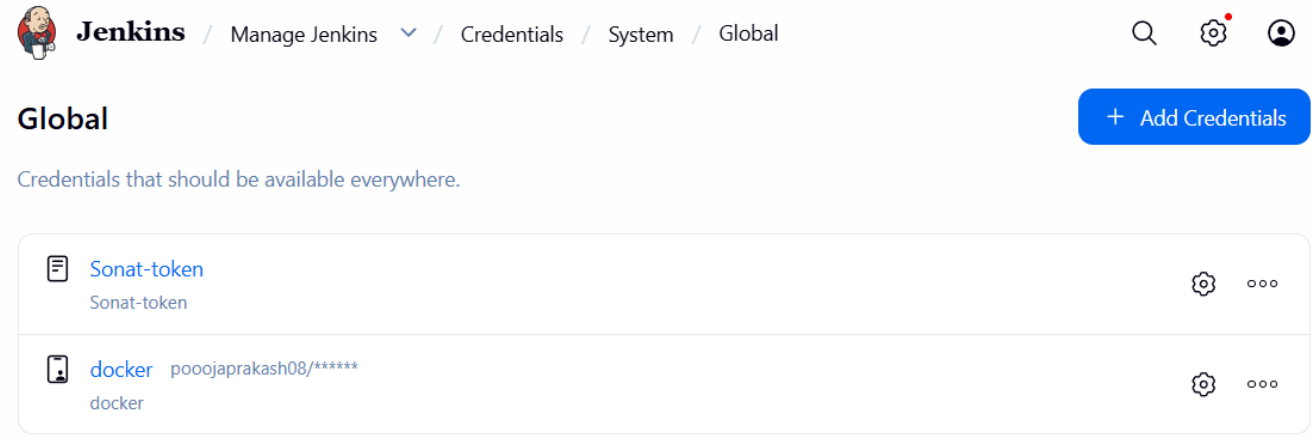
- Installed Trivy for file system vulnerability scanning and SonarQube for code quality analysis.



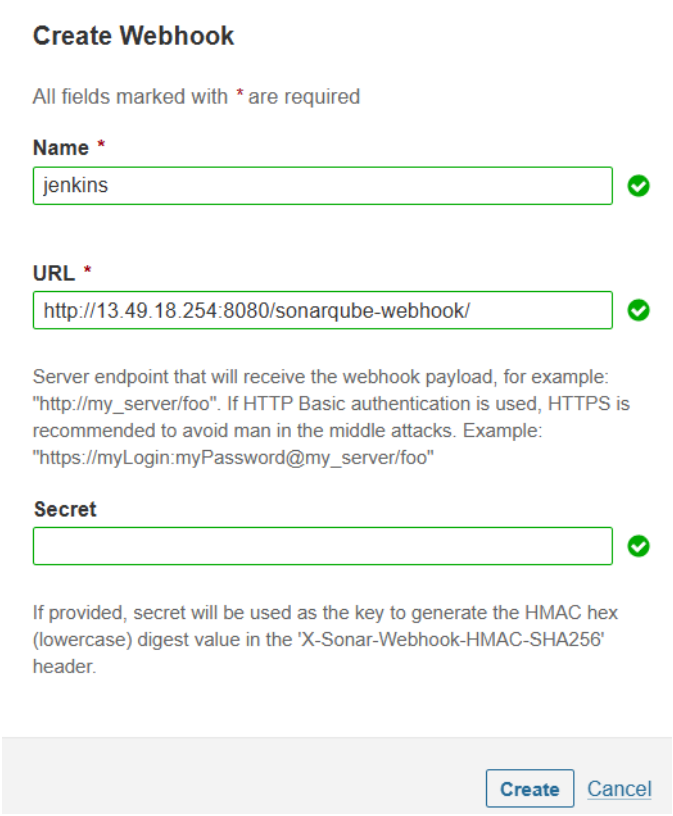
- Accessed:
  - ✓ Jenkins via port 8080
  - ✓ SonarQube via port 9000

- Generated a SonarQube authentication token and added it to Jenkins credentials.
- Also added Docker Hub credentials in Jenkins.

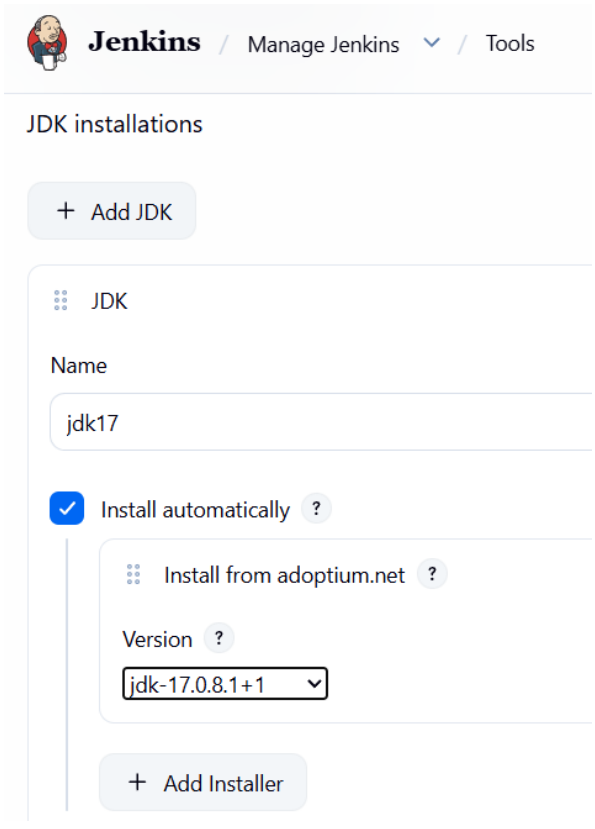


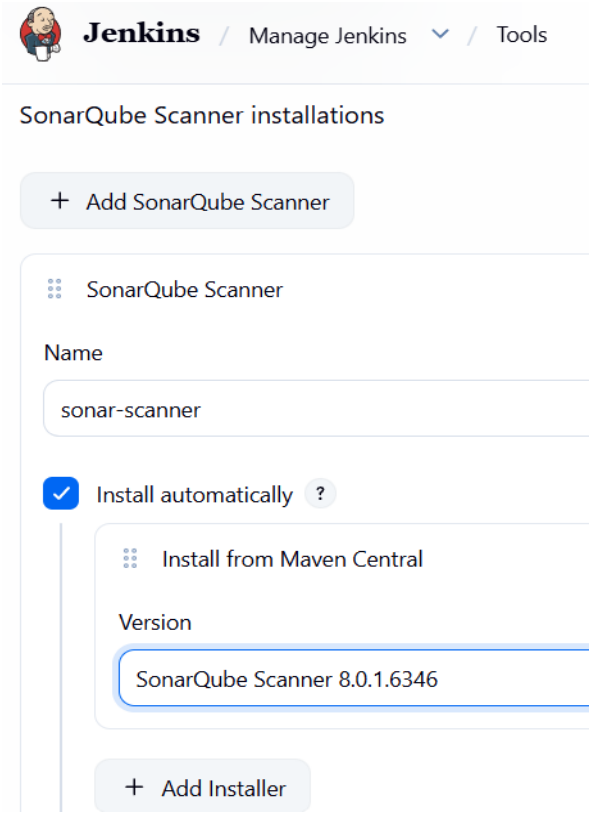


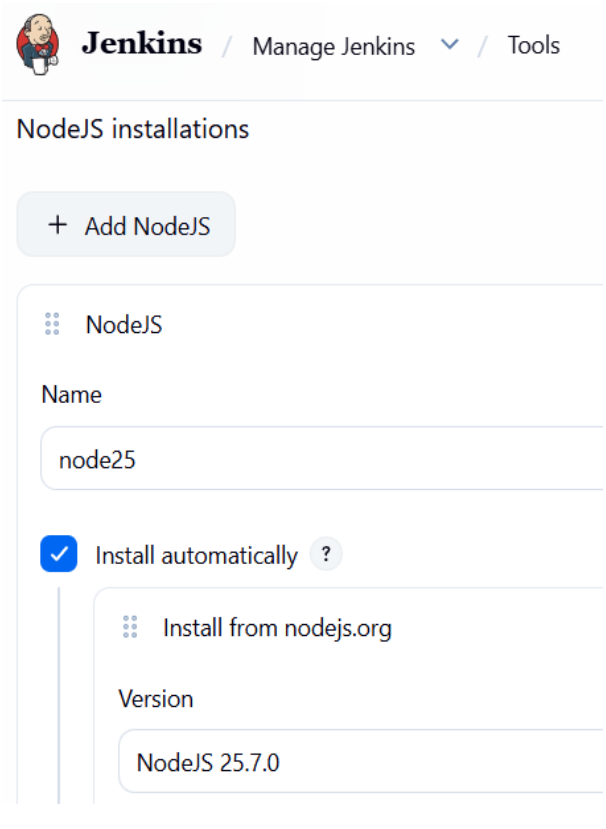
- Created a webhook in SonarQube and configured it with the Jenkins URL to trigger pipeline updates automatically.

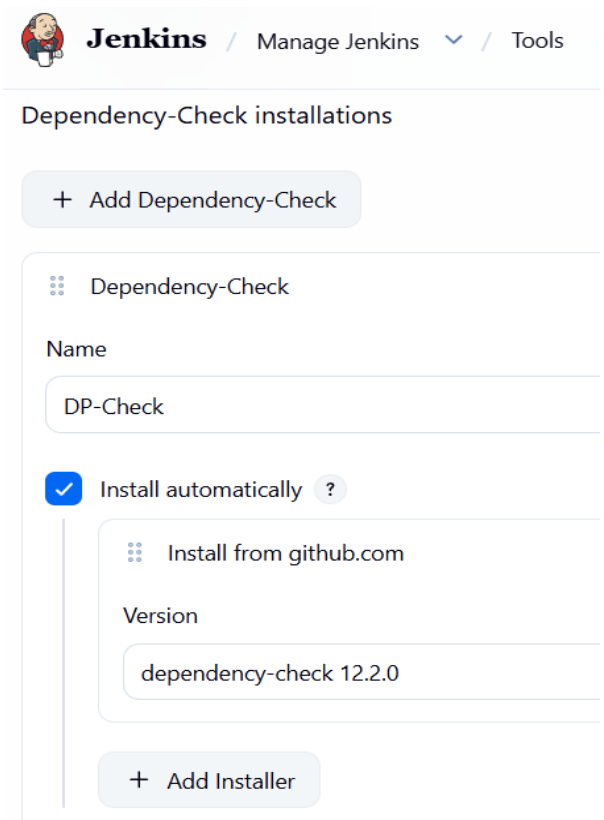


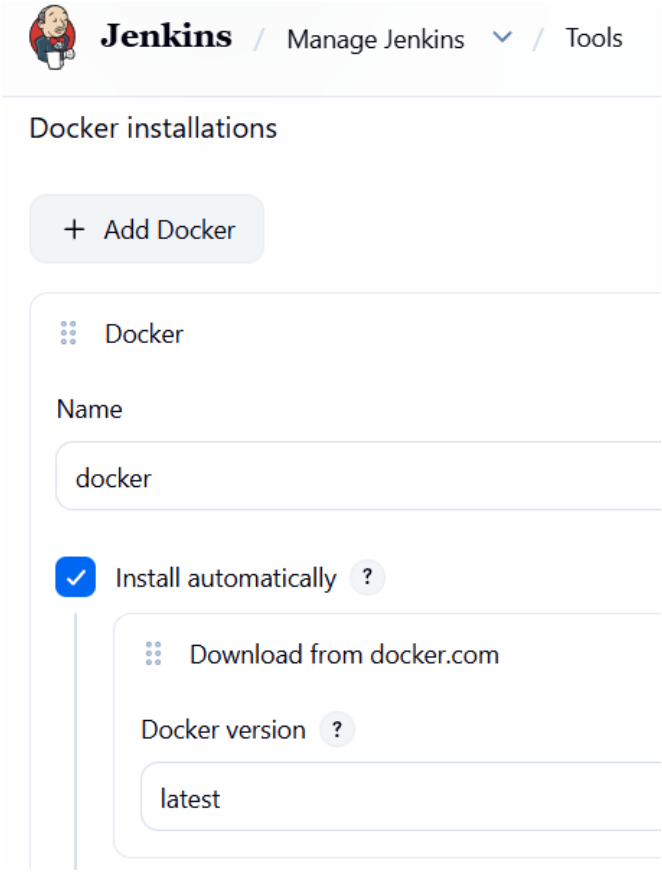
- Configured required tools in Jenkins: JDK, SonarQube Scanner, NodeJS, OWASP Dependency Check and Docker.



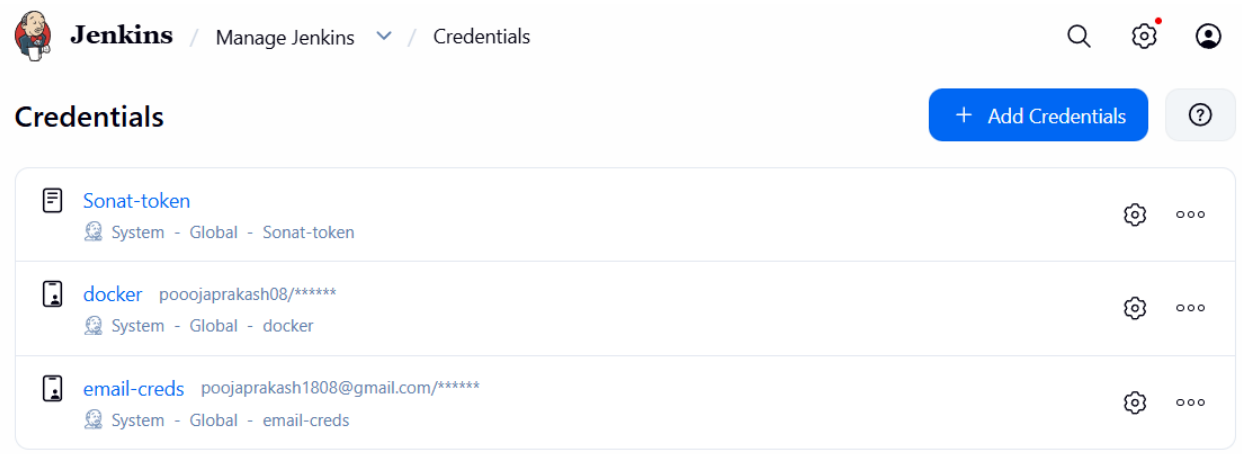



Jenkins / Manage Jenkins / Tools
NodeJS installations
+ Add NodeJS
NodeJS
Name
node25
Install automatically
Install from nodejs.org
Version
NodeJS 25.7.0

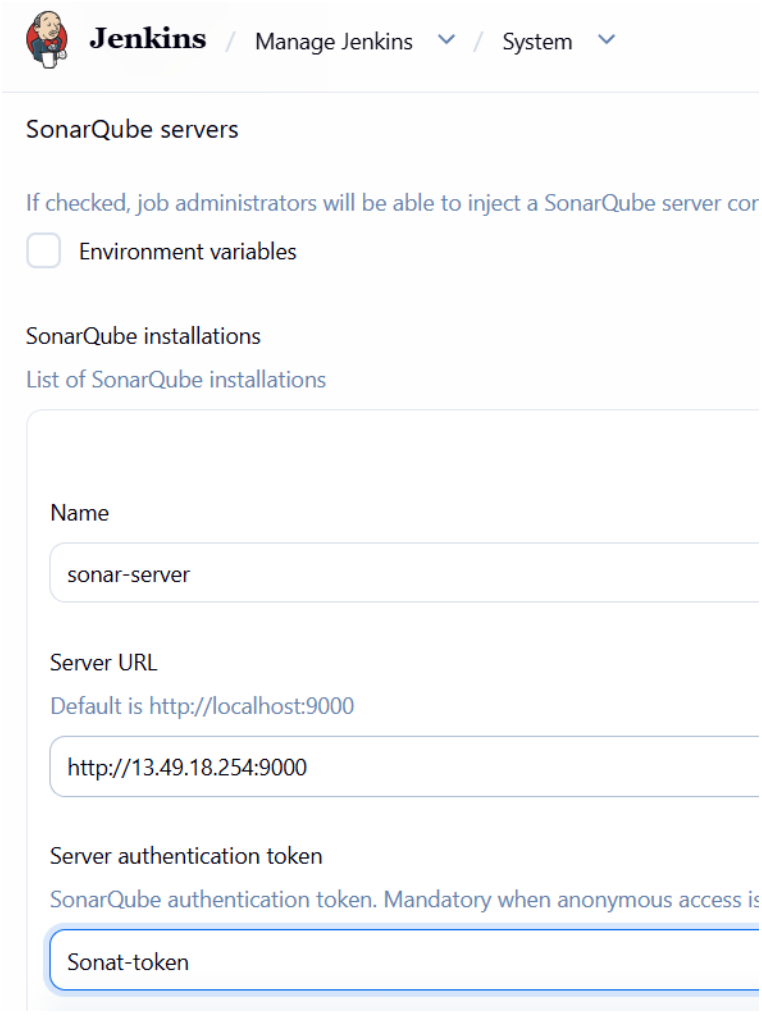

Jenkins / Manage Jenkins / Tools
Dependency-Check installations
+ Add Dependency-Check
Dependency-Check
Name
DP-Check
Install automatically
Install from github.com
Version
dependency-check 12.2.0
+ Add Installer

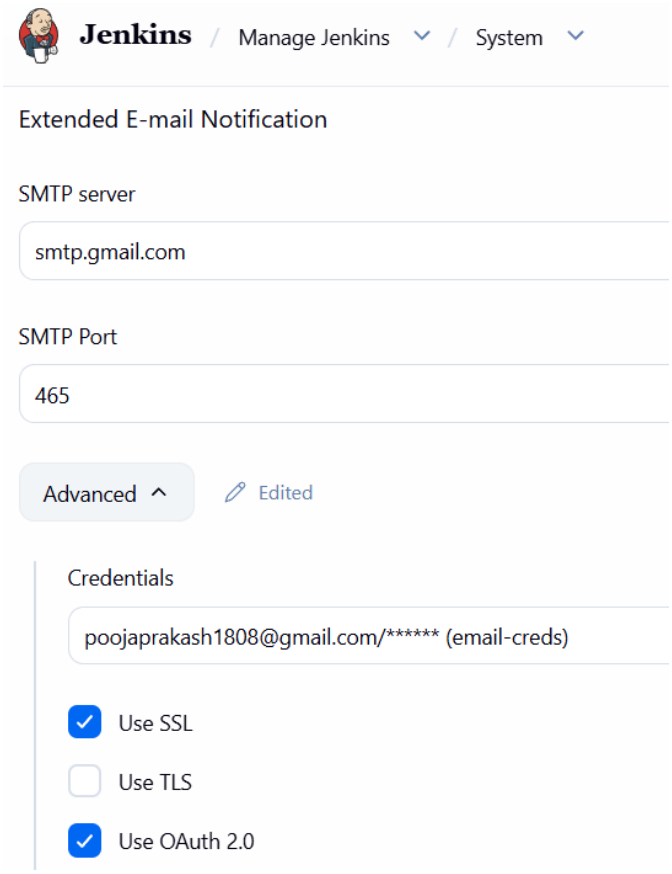

Jenkins / Manage Jenkins / Tools
Docker installations
+ Add Docker
Docker
Name
docker
Install automatically
Download from docker.com
Docker version
latest

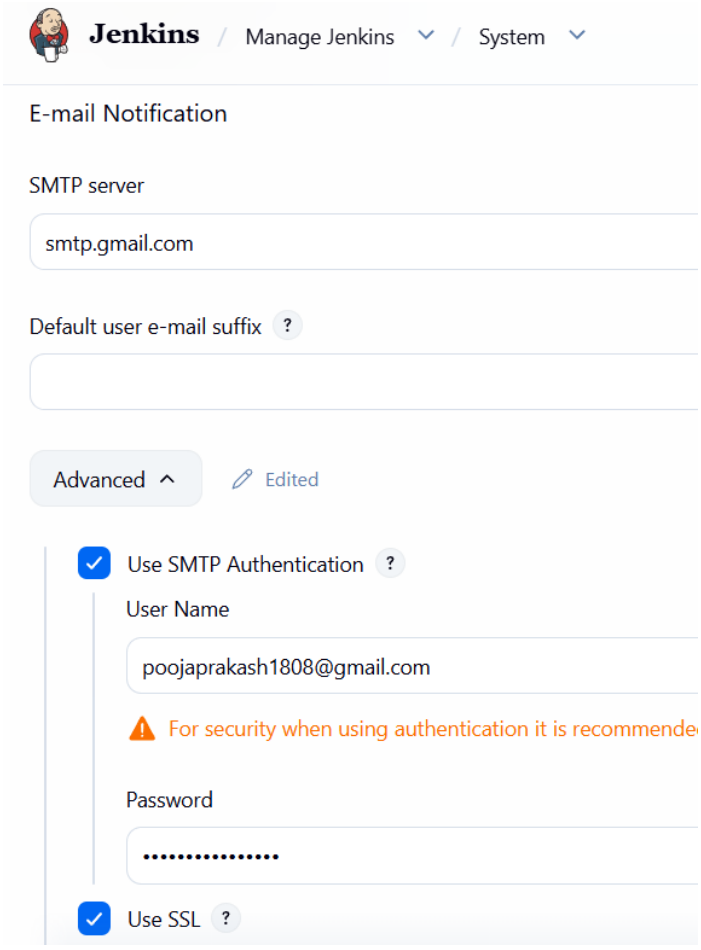
- Configured email credentials in Jenkins to send notifications for build success, failures, and updates.

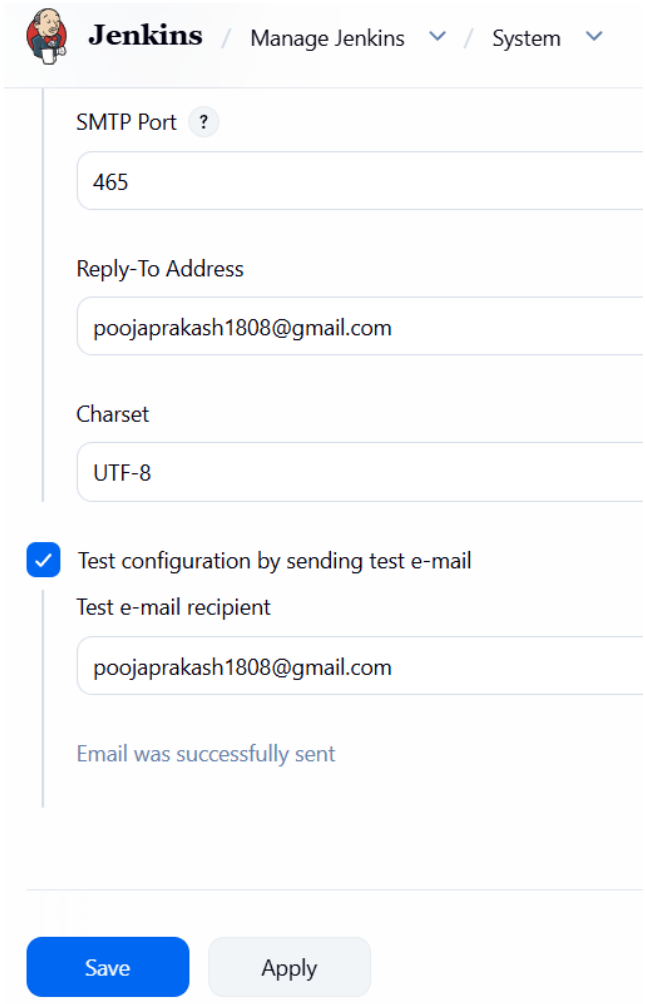


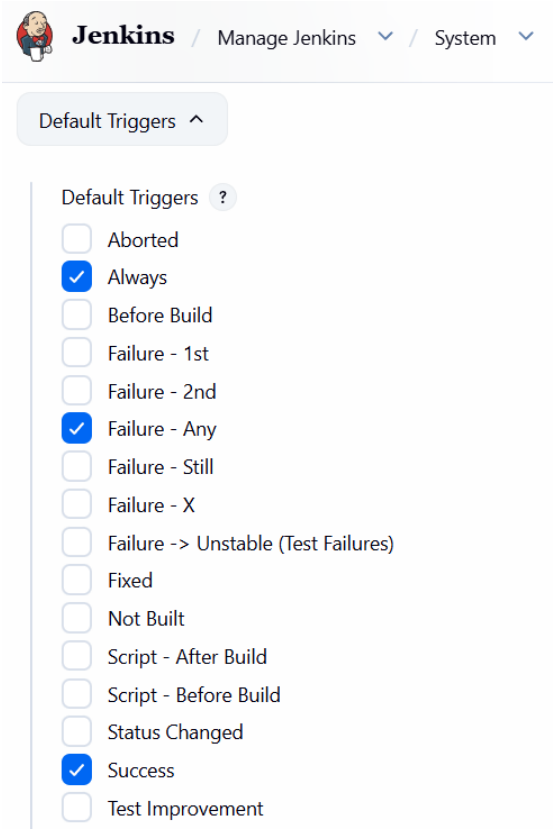
- Completed system configuration for SonarQube and email notifications in Jenkins.




Jenkins / Manage Jenkins / System
Extended E-mail Notification
SMTP server
smtp.gmail.com
SMTP Port
465
Advanced
Edited
Credentials
poojaprakash1808@gmail.com/****** (email-creds)
Use SSL
Use TLS
Use OAuth 2.0

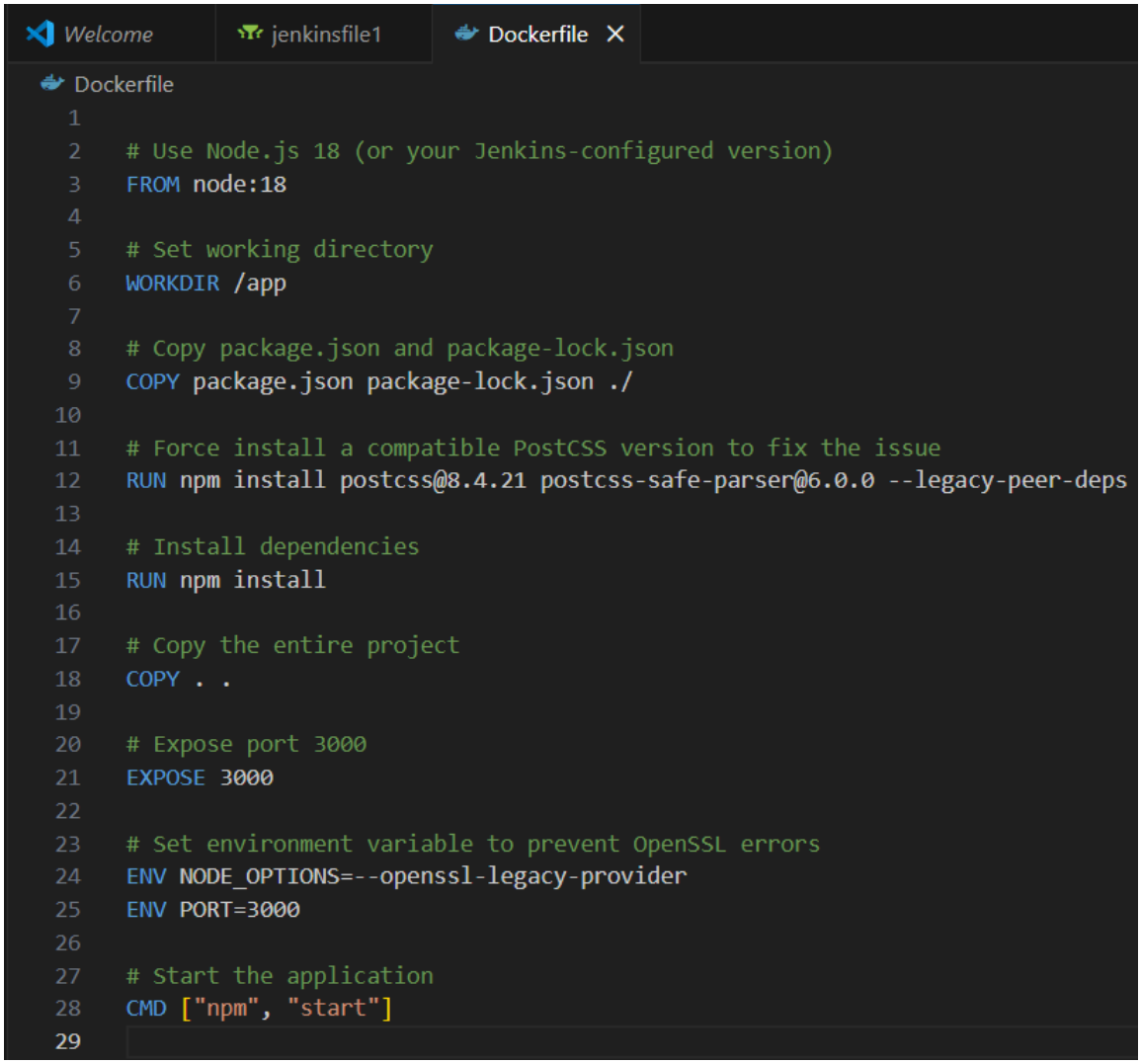

Jenkins / Manage Jenkins / System
E-mail Notification
SMTP server
smtp.gmail.com
Default user e-mail suffix
Advanced
Edited
Use SMTP Authentication
User Name
poojaprakash1808@gmail.com
For security when using authentication it is recommende
Password
Use SSL


Jenkins / Manage Jenkins / System
SMTP Port ?
465
Reply-To Address
poojaprakash1808@gmail.com
Charset
UTF-8
Test configuration by sending test e-mail
Test e-mail recipient
poojaprakash1808@gmail.com
Email was successfully sent
Save
Apply


Jenkins / Manage Jenkins / System
Default Triggers
Default Triggers ?
Aborted
Always
Before Build
Failure - 1st
Failure - 2nd
Failure - Any
Failure - Still
Failure - X
Failure -> Unstable (Test Failures)
Fixed
Not Built
Script - After Build
Script - Before Build
Status Changed
Success
Test Improvement

Phase 2: CI/CD Pipeline Execution

- Created a Dockerfile to containerize the application.



- Created a Jenkinsfile that includes the following stages: Clone source code from GitHub, Perform SonarQube code analysis, install npm dependencies, Run OWASP dependency check, Perform Trivy file system scan, Build Docker image, Tag and push image to Docker Hub, Run Docker Scout image analysis, Deploy containerized application and Send email notifications.

Welcome | jenkinsfile1 | Dockerfile

jenkinsfile1

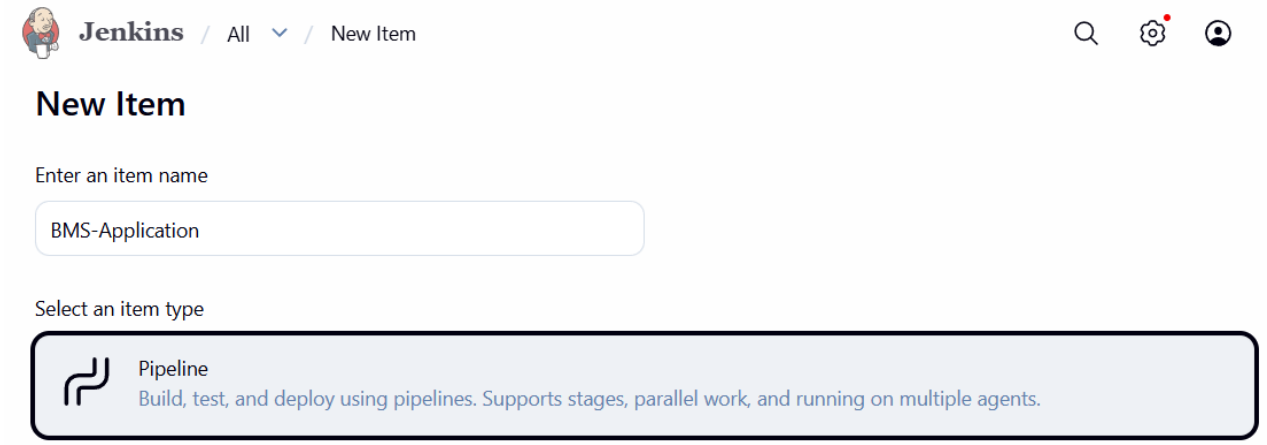
```
pipeline {
    agent any
    tools {
        jdk 'jdk17'
        nodejs 'node25'
    }
    environment {
        SCANNER_HOME = tool 'sonar-scanner'
    }
    stages {
        stage('Clean Workspace') {
            steps {
                cleanWs()
            }
        }
        stage('Checkout from Git') {
            steps {
                git branch: 'main', url: 'https://github.com/PoojaPrakash08/Book-My-Show.git'
                sh 'ls -la'  // Verify files after checkout
            }
        }
        stage('SonarQube Analysis') {
            steps {
                withSonarQubeEnv('sonar-server') {
                    sh '''
                    $SCANNER_HOME/bin/sonar-scanner -Dsonar.projectName=BMS \
                    -Dsonar.projectKey=BMS
                    '''
                }
            }
        }
        stage('Quality Gate') {
            steps {
                script {
                    waitForQualityGate abortPipeline: false, credentialsId: 'Sonar-token'
                }
            }
        }
```

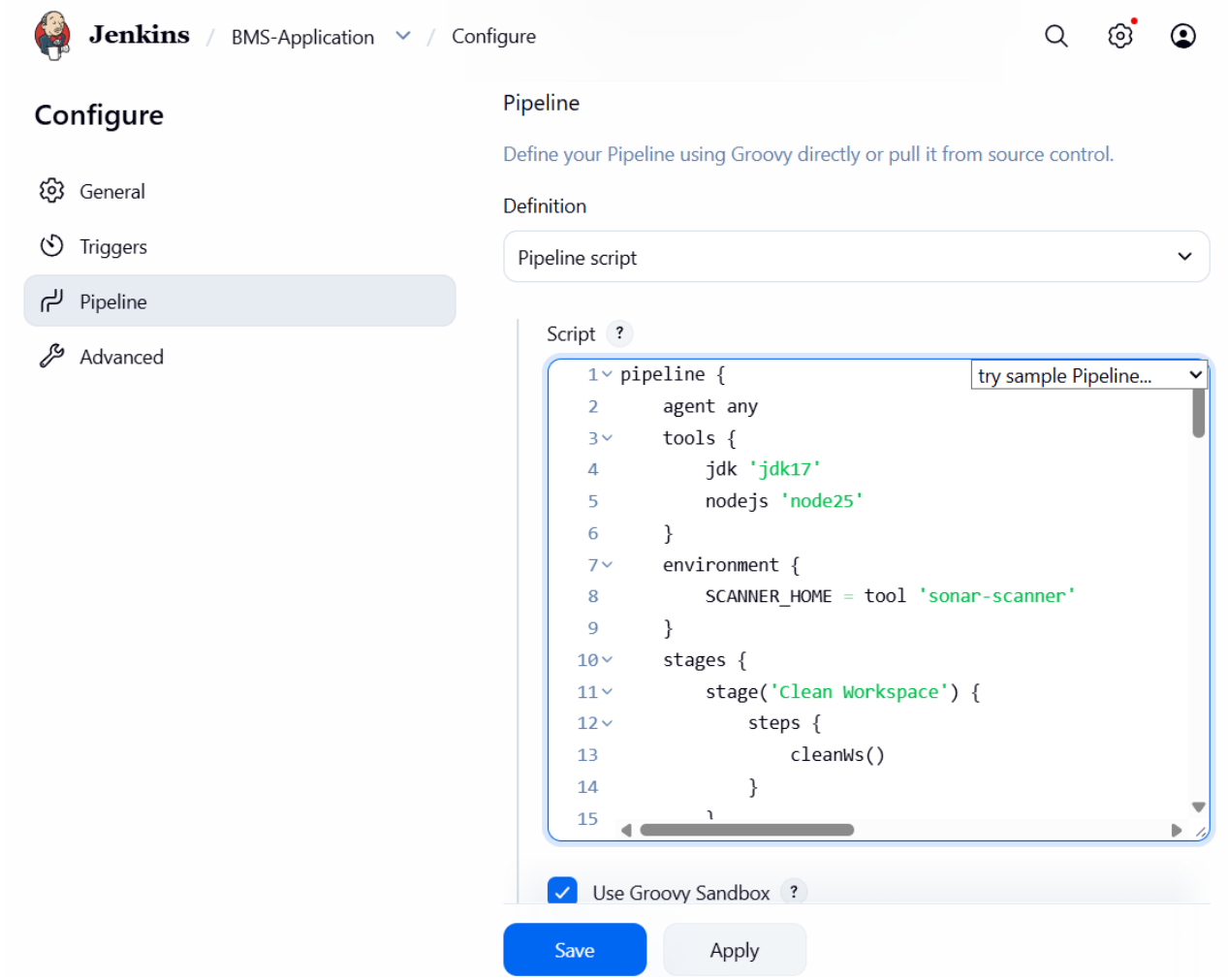
```
    }
    stage('Install Dependencies') {
        steps {
            sh '''
            cd bookmyshow-app
            ls -la  # Verify package.json exists
            if [ -f package.json ]; then
                rm -rf node_modules package-lock.json  # Remove old dependencies
                npm install  # Install fresh dependencies
            else
                echo "Error: package.json not found in bookmyshow-app!"
                exit 1
            fi
            '''
        }
    }
    stage('OWASP FS Scan') {
        steps {
            dependencyCheck additionalArguments: '--scan ./ --disableYarnAudit --disableNodeAudit', odcInstallation: 'DP-Check'
            dependencyCheckPublisher pattern: '**/dependency-check-report.xml'
        }
    }
    stage('Trivy FS Scan') {
        steps {
            sh 'trivy fs . > trivyfs.txt'
        }
    }
```

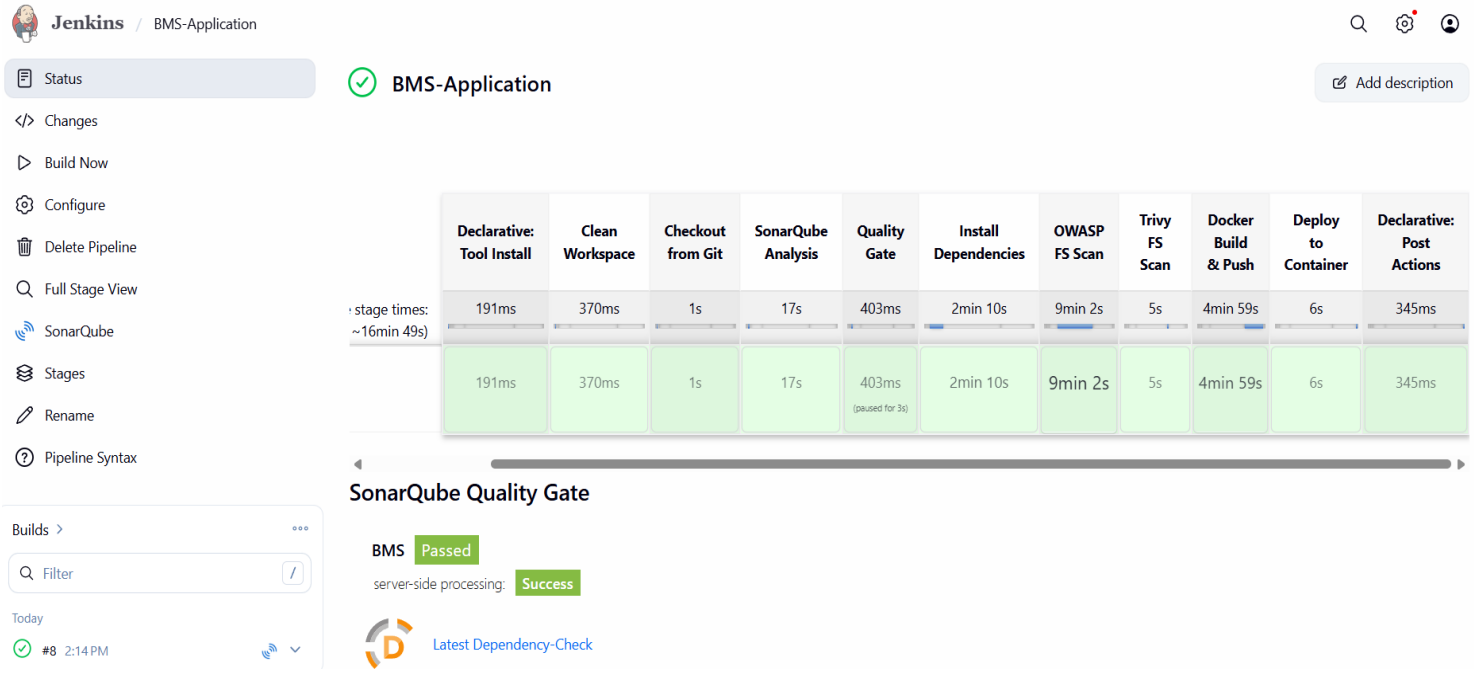
```
        stage('Docker Build & Push') {
            steps {
                script {
                    withDockerRegistry(credentialsId: 'docker', toolName: 'docker') {
                        sh '''
                        echo "Building Docker image..."
                        docker build --no-cache -t poojaprakash08/bms:latest -f bookmyshow-app/Dockerfile bookmyshow-app

                        echo "Pushing Docker image to registry..."
                        docker push poojaprakash08/bms:latest
                        '''
                    }
                }
            }
        }
        stage('Deploy to Container') {
            steps {
                sh '''
                echo "Stopping and removing old container..."
                docker stop bms || true
                docker rm bms || true

                echo "Running new container on port 3000..."
                docker run -d --restart=always --name bms -p 3000:3000 poojaprakash08/bms:latest

                echo "Checking running containers..."
                docker ps -a

                echo "Fetching logs..."
                sleep 5  # Give time for the app to start
                docker logs bms
                '''
            }
        }
```

```
        }
        post {
            always {
                emailext attachLog: true,
                    subject: "'${currentBuild.result}'",
                    body: "Project: ${env.JOB_NAME}<br/>" +
                          "Build Number: ${env.BUILD_NUMBER}<br/>" +
                          "URL: ${env.BUILD_URL}<br/>",
                    to: 'poojaprakash1808@gmail.com',
                    attachmentsPattern: 'trivyfs.txt,trivyimage.txt'
            }
        }
    }
```

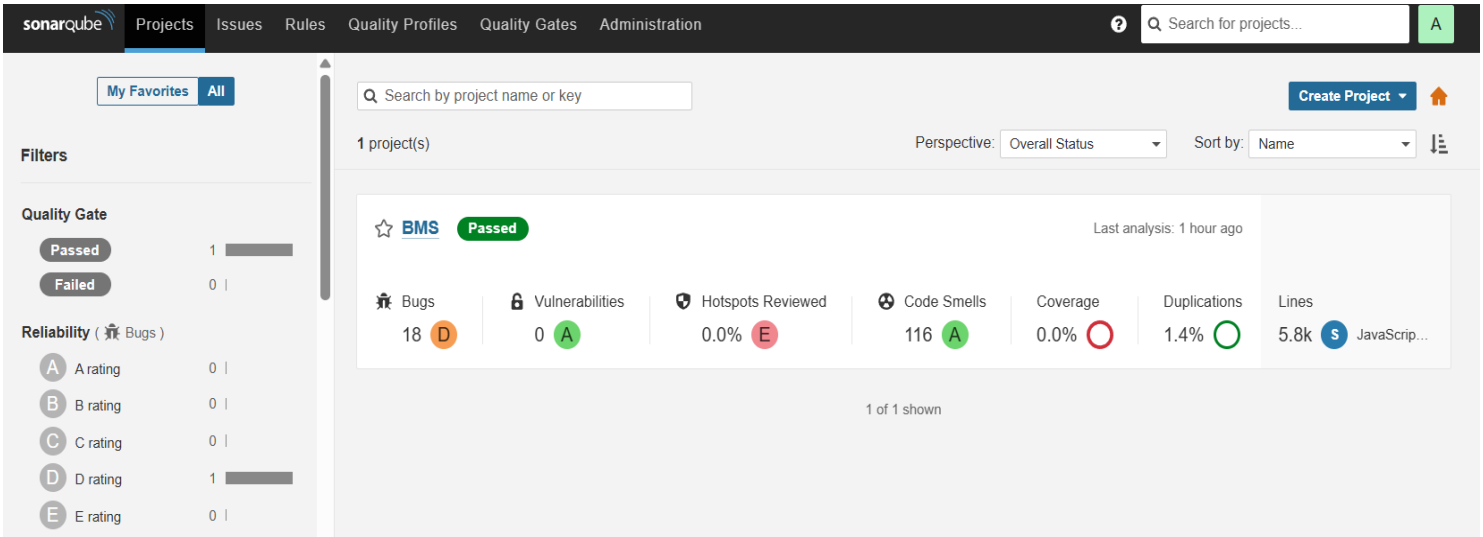
- Created a pipeline job named **BMS-Application** and configured it with the pipeline script.



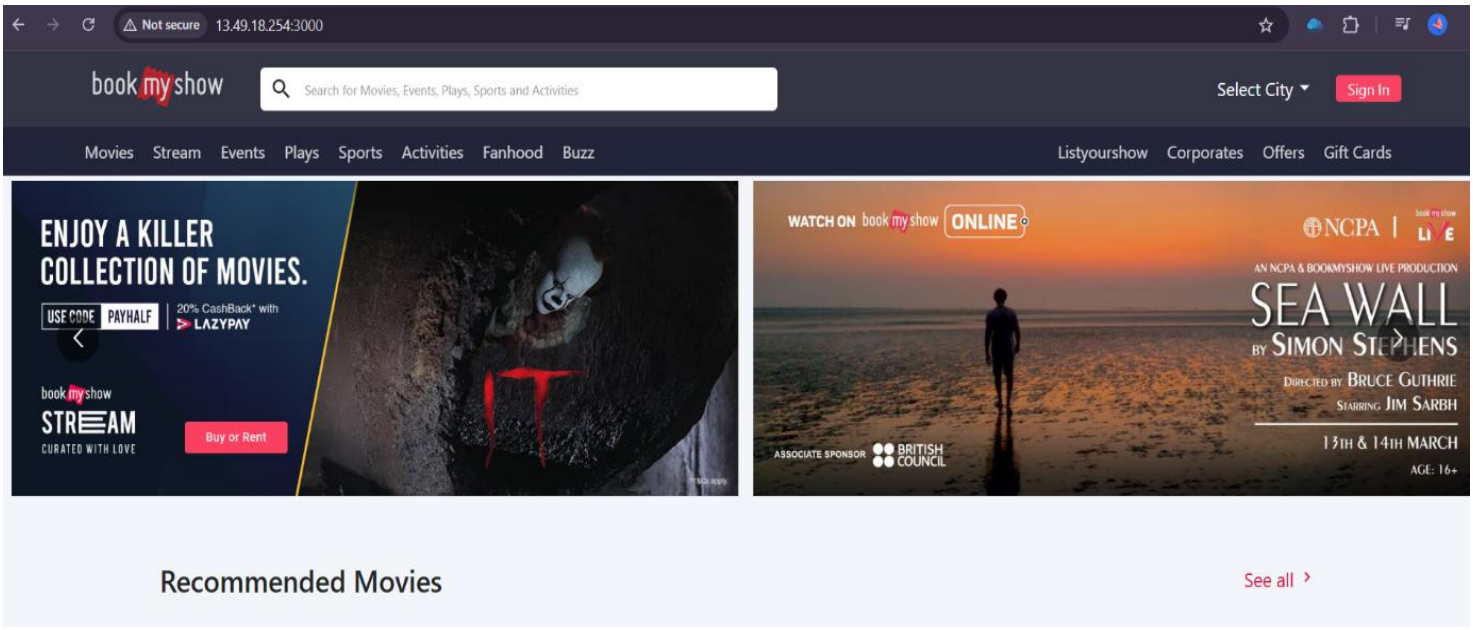




- Successfully generated the SonarQube analysis report after pipeline execution.

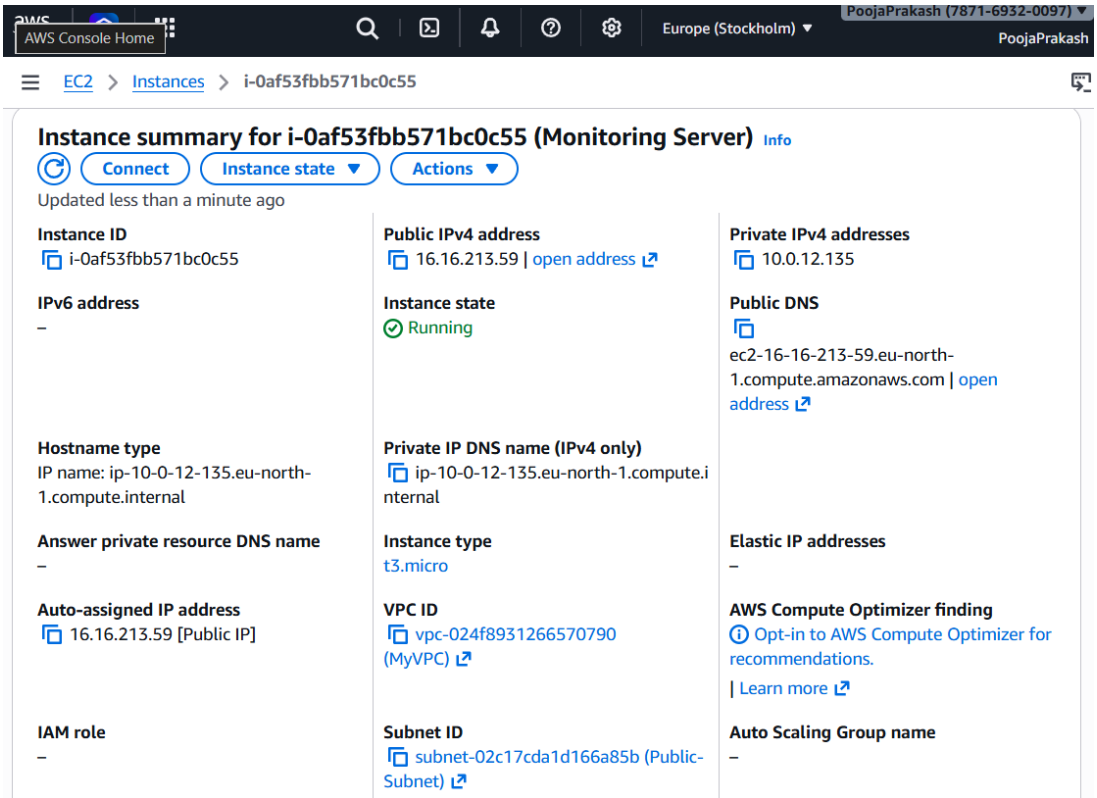


- Accessed the Book My Show application using the public IP of the BMS-Server on port 3000.

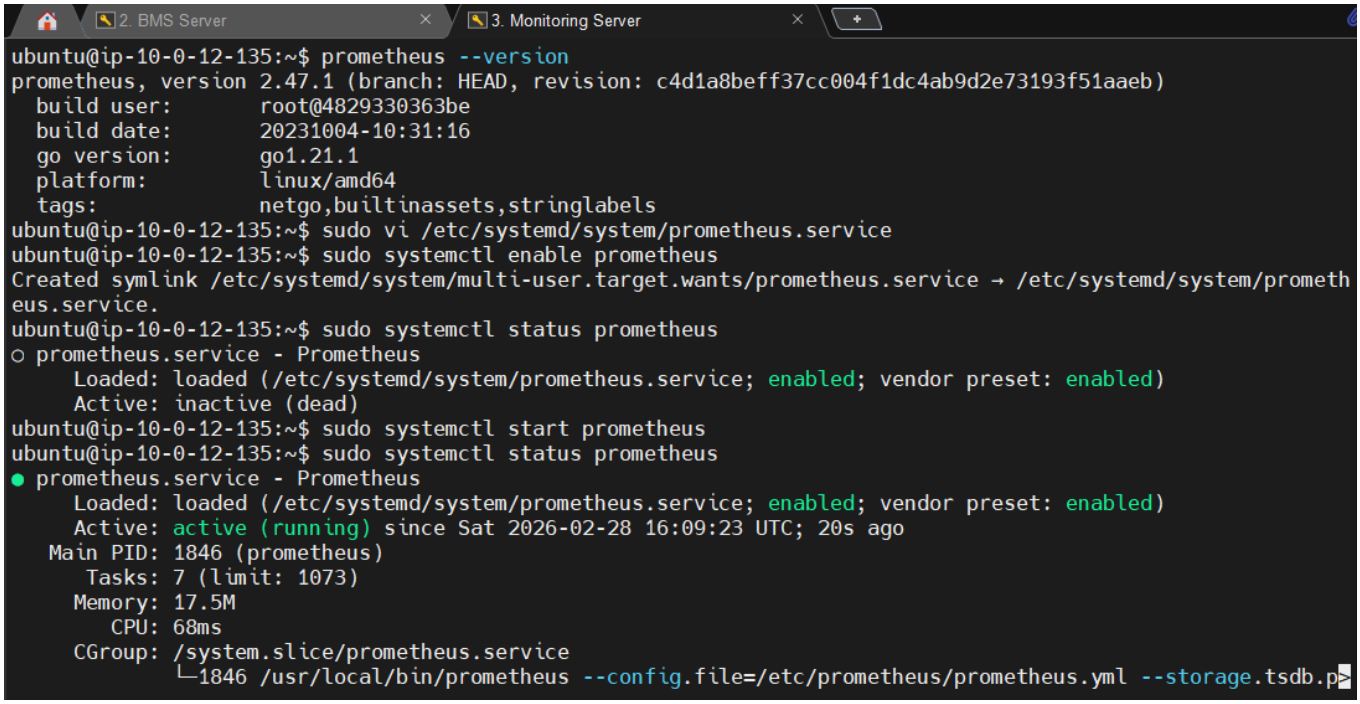


Phase 3: Monitoring Setup

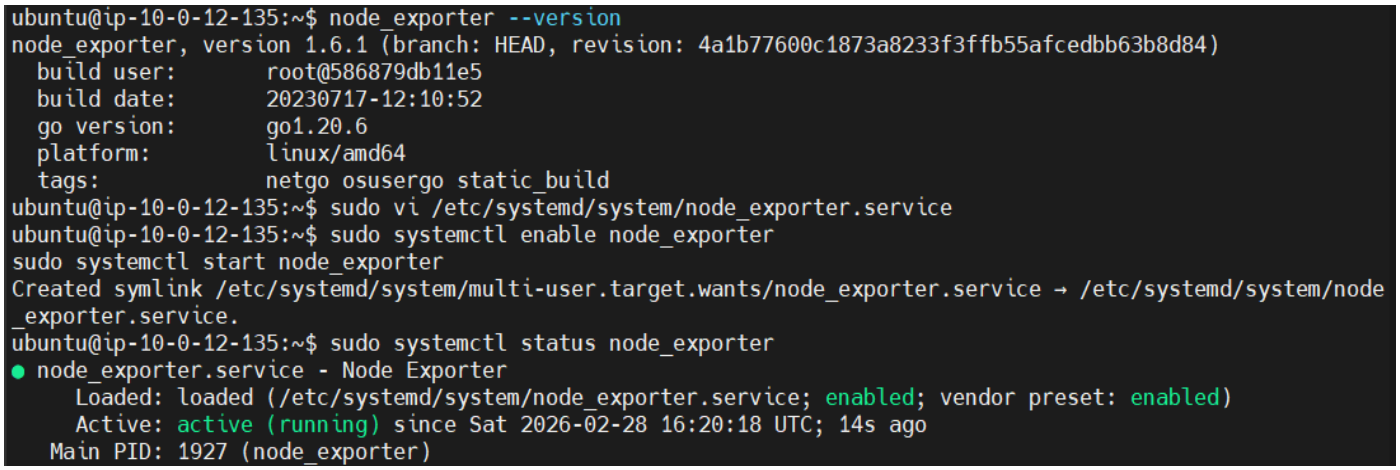
- Launched another Ubuntu EC2 instance named **Monitoring Server**.



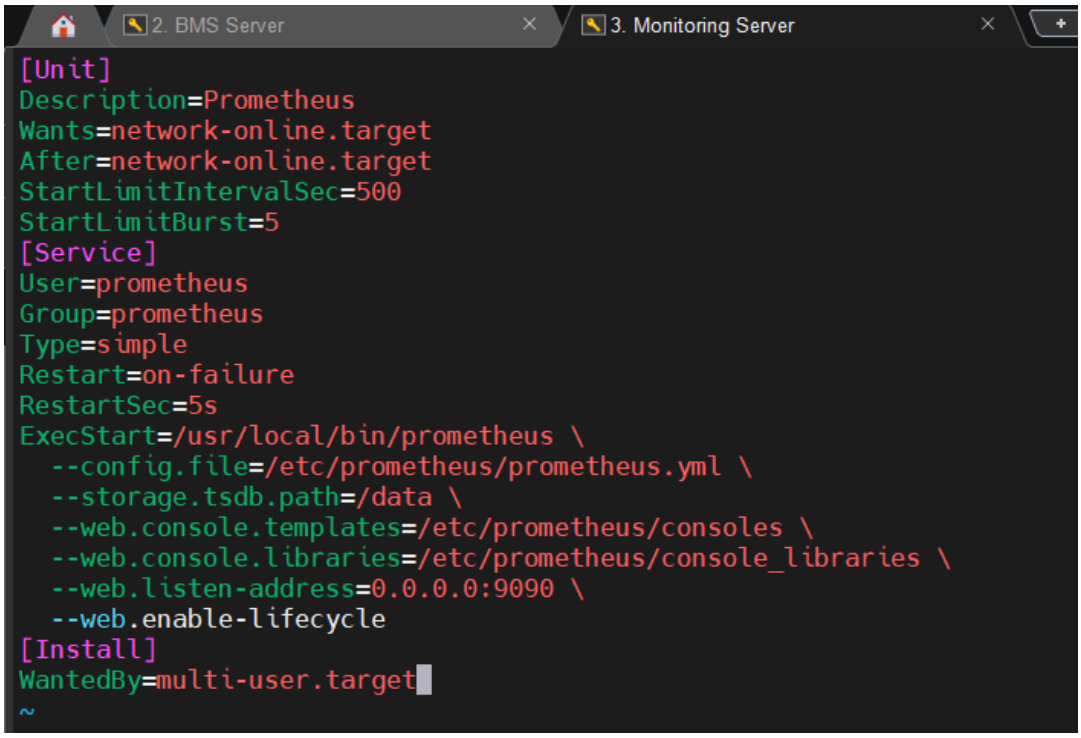
- Installed Prometheus on the Monitoring Server.



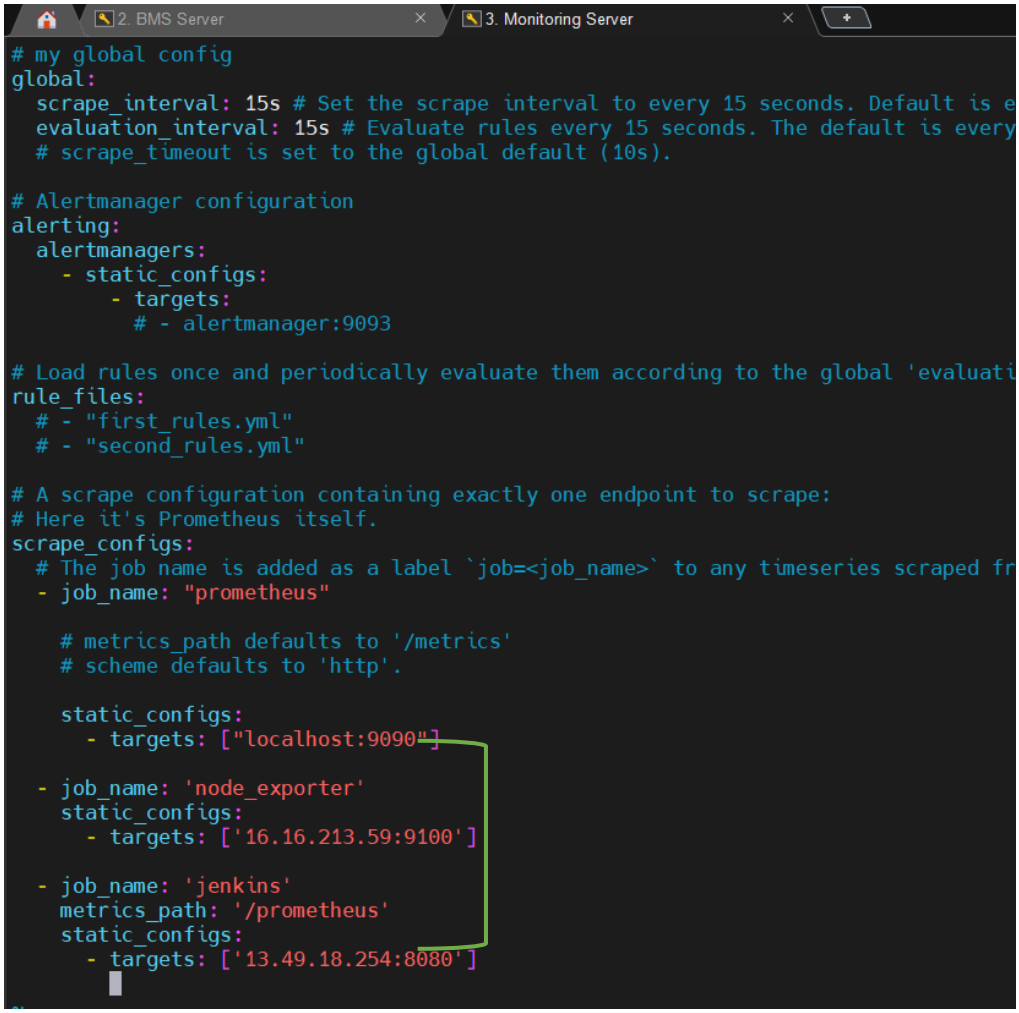
- Installed Node Exporter to collect system-level metrics.

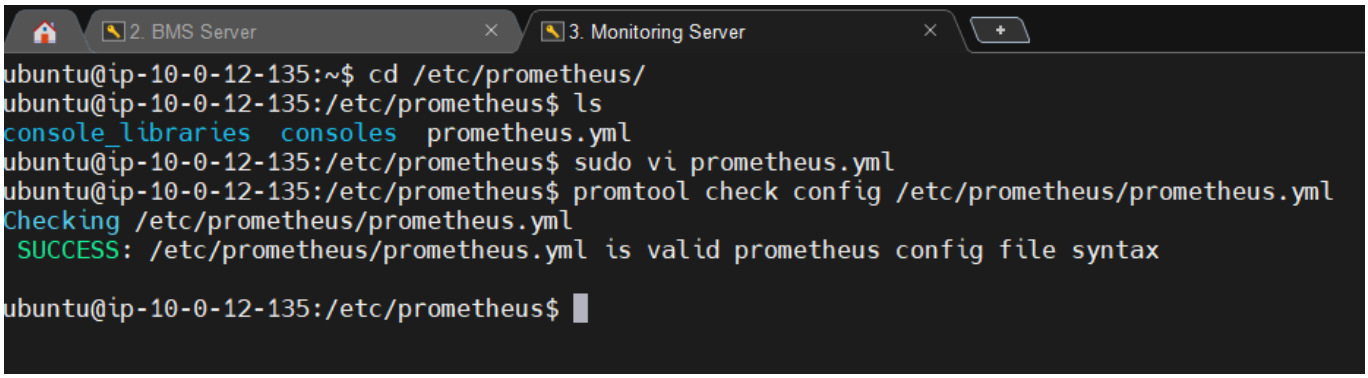


- Configured Prometheus to scrape metrics from Node Exporter and Jenkins.

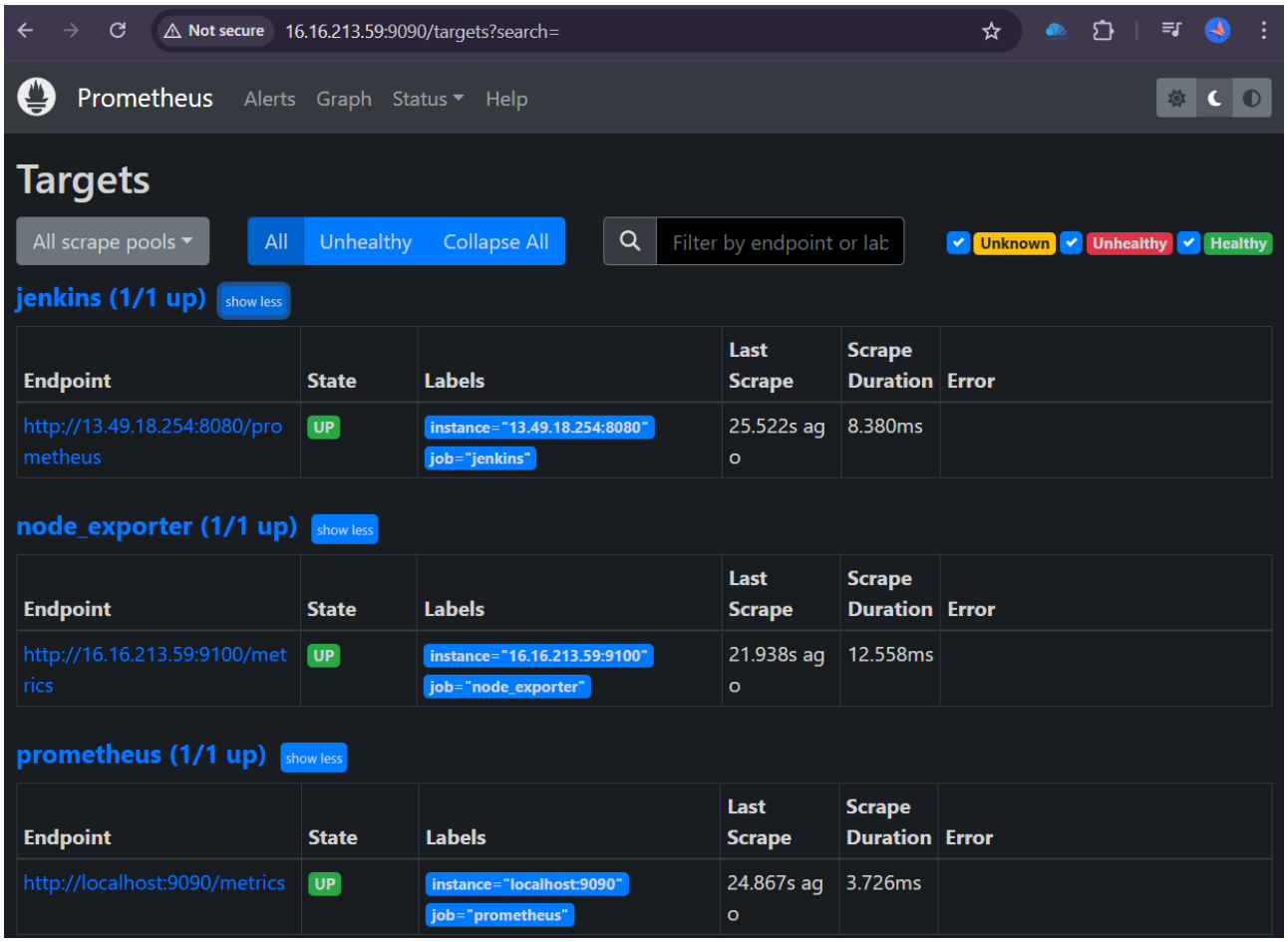


- Updated and verified the syntax of the prometheus.yml configuration file.

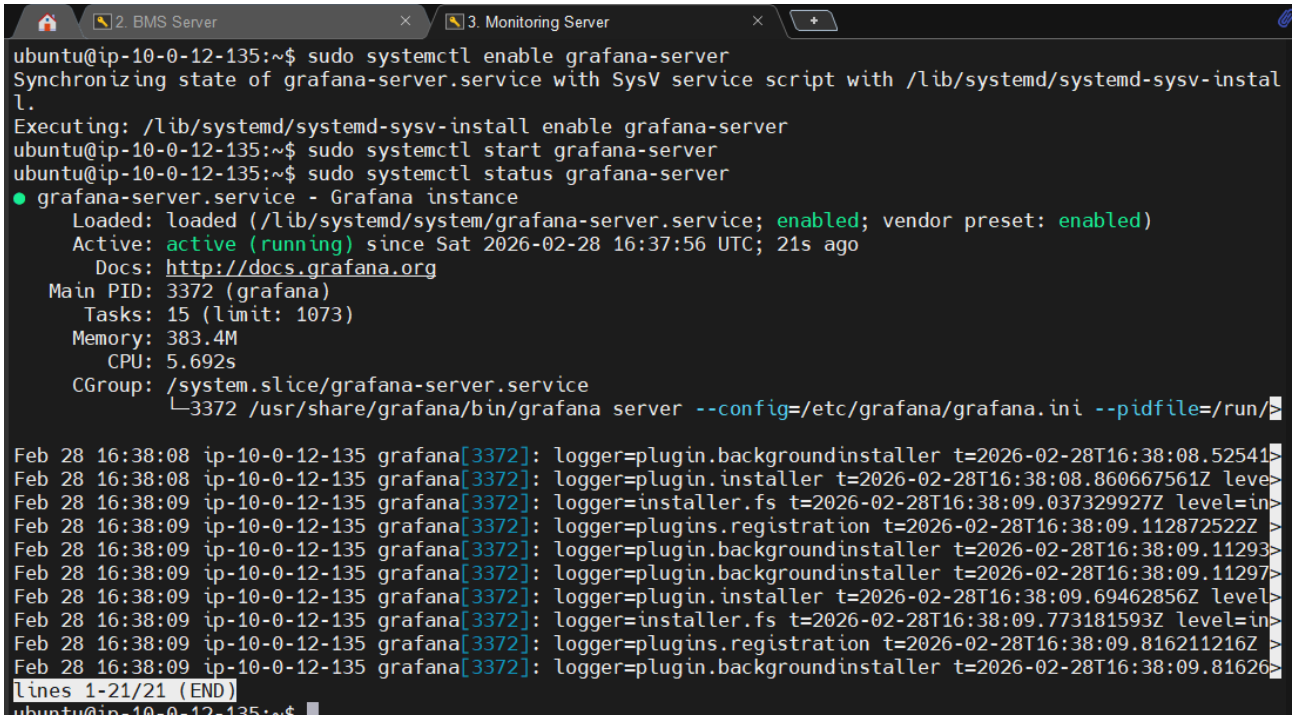




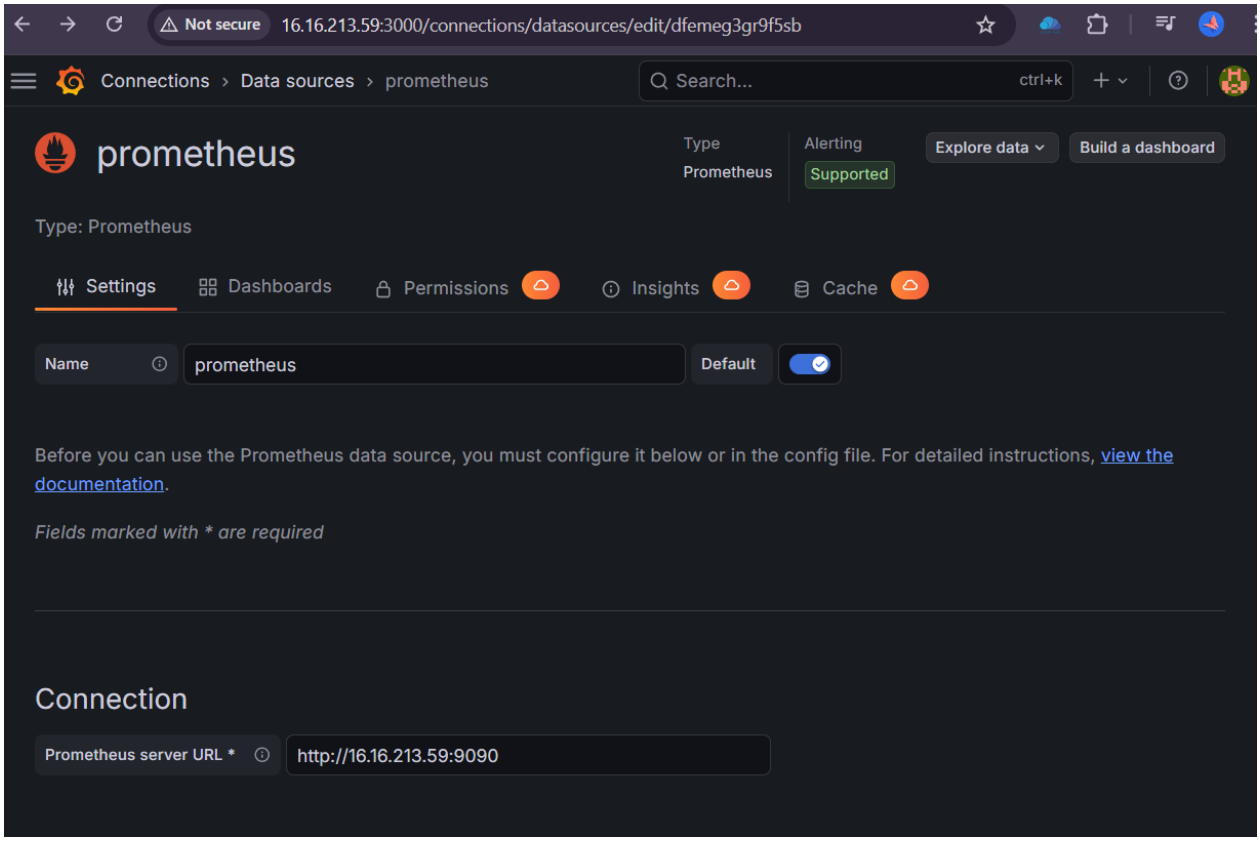
- Accessed Prometheus via port 9090 and verified that all targets were in UP state.

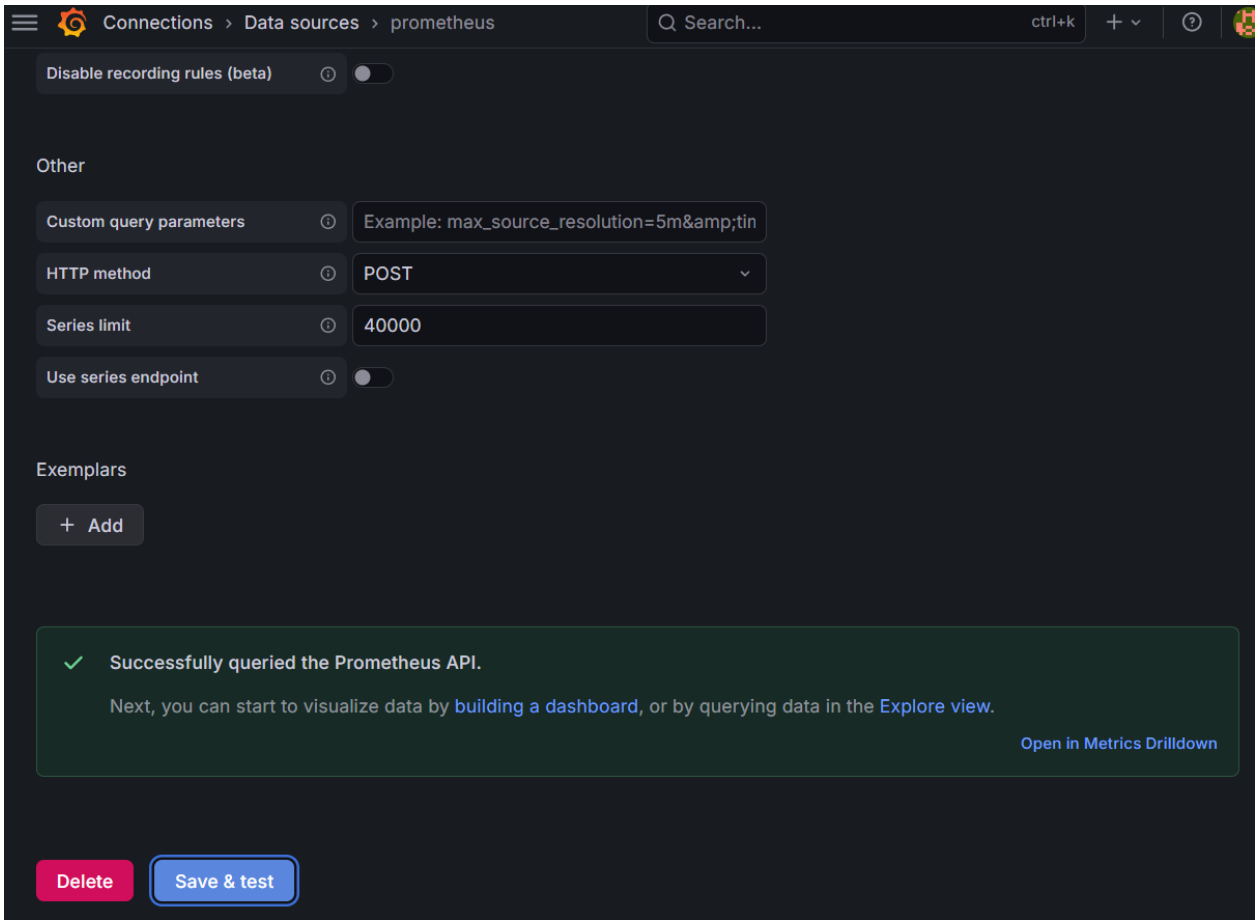


- Installed and started Grafana on the Monitoring Server.

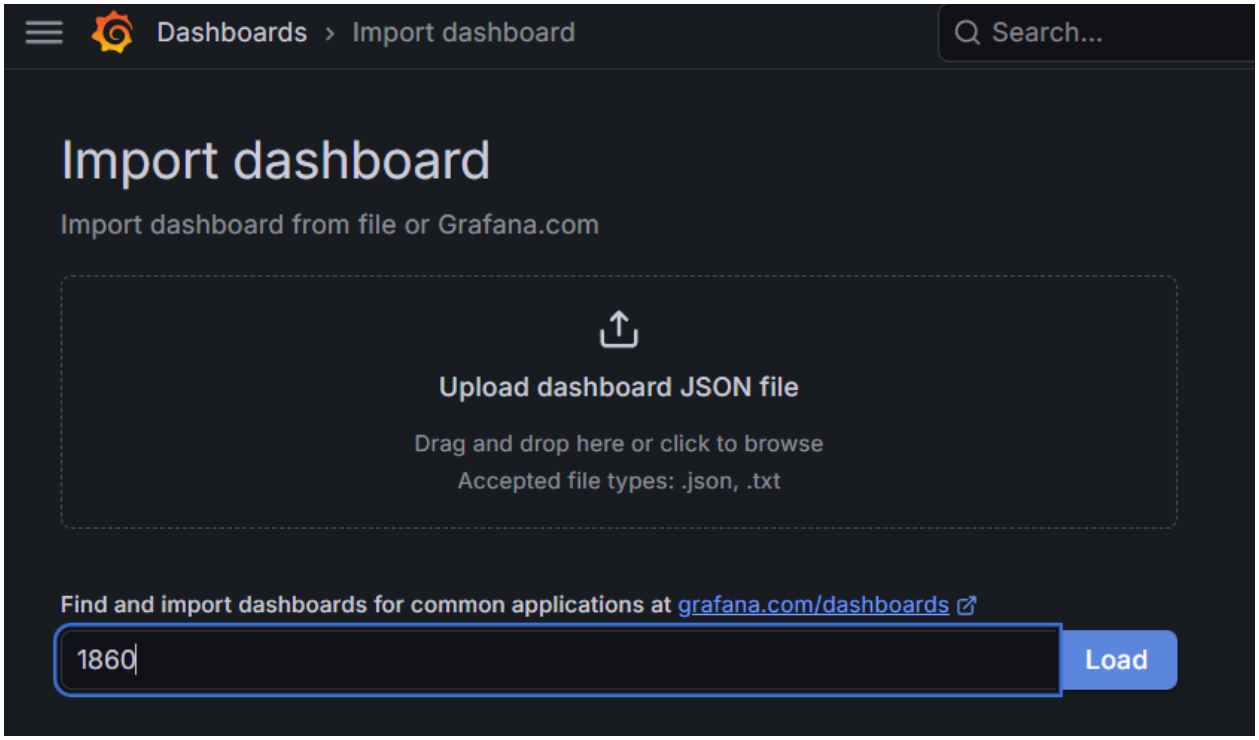


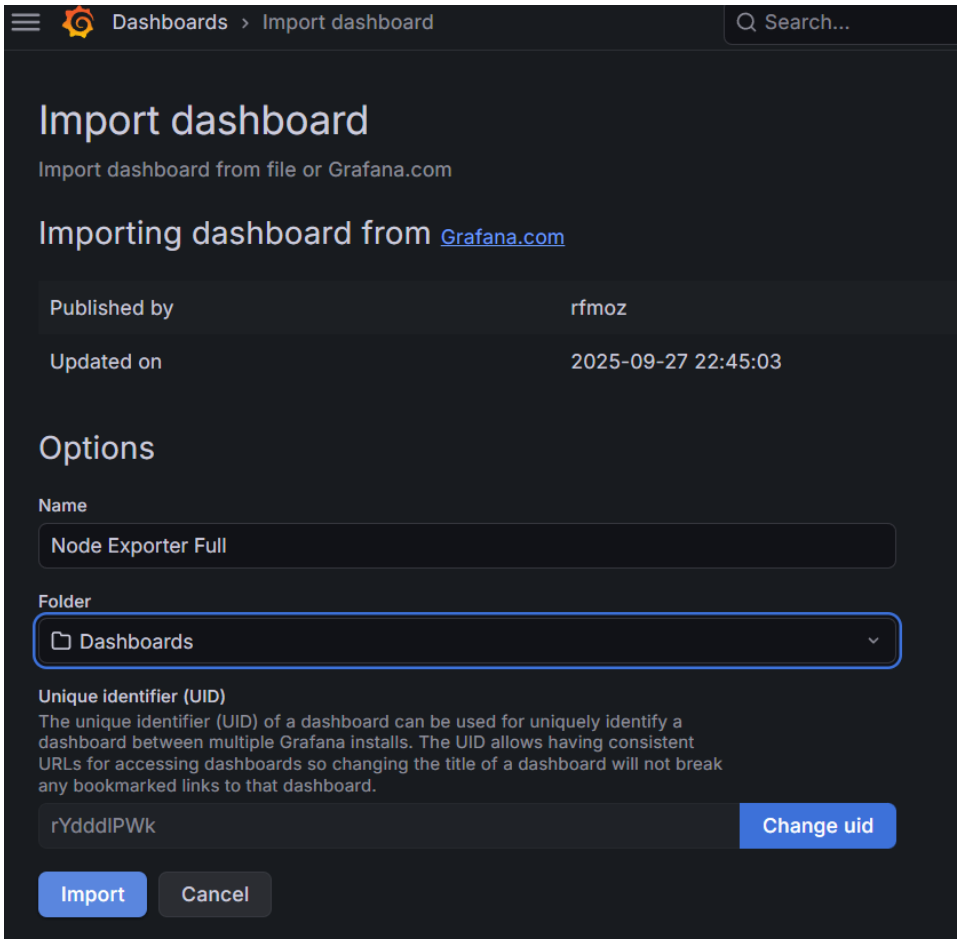
- Accessed Grafana via port 3000 and added Prometheus as a data source.

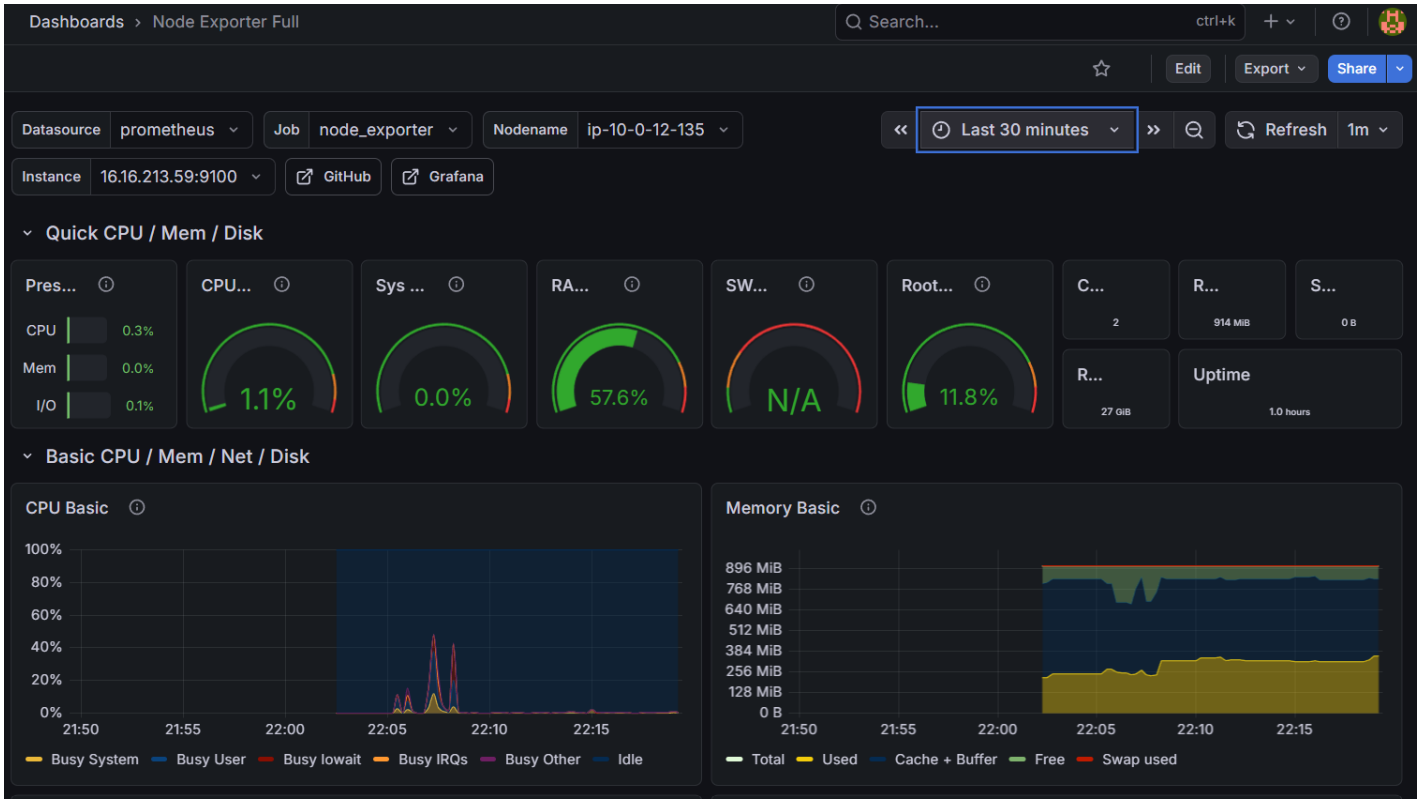




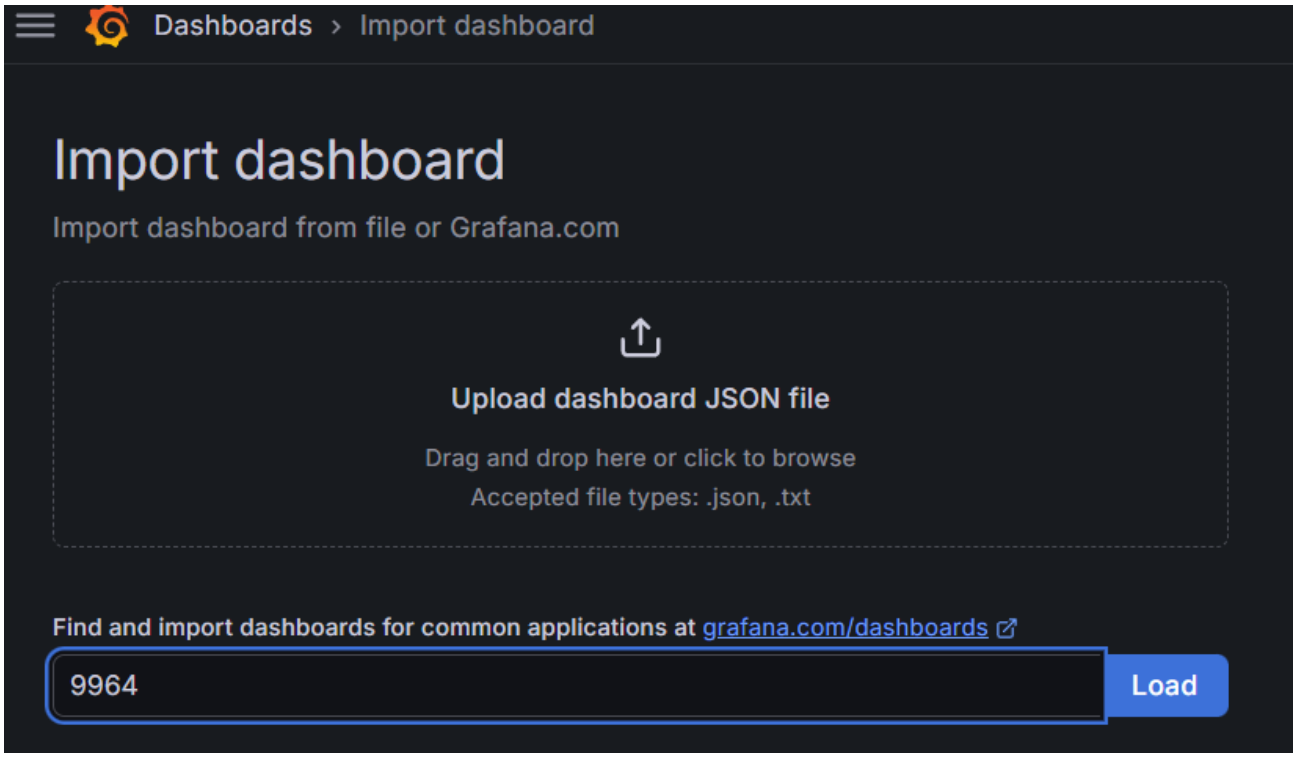
- Imported the Node Exporter dashboard using Grafana Dashboard ID 1860.

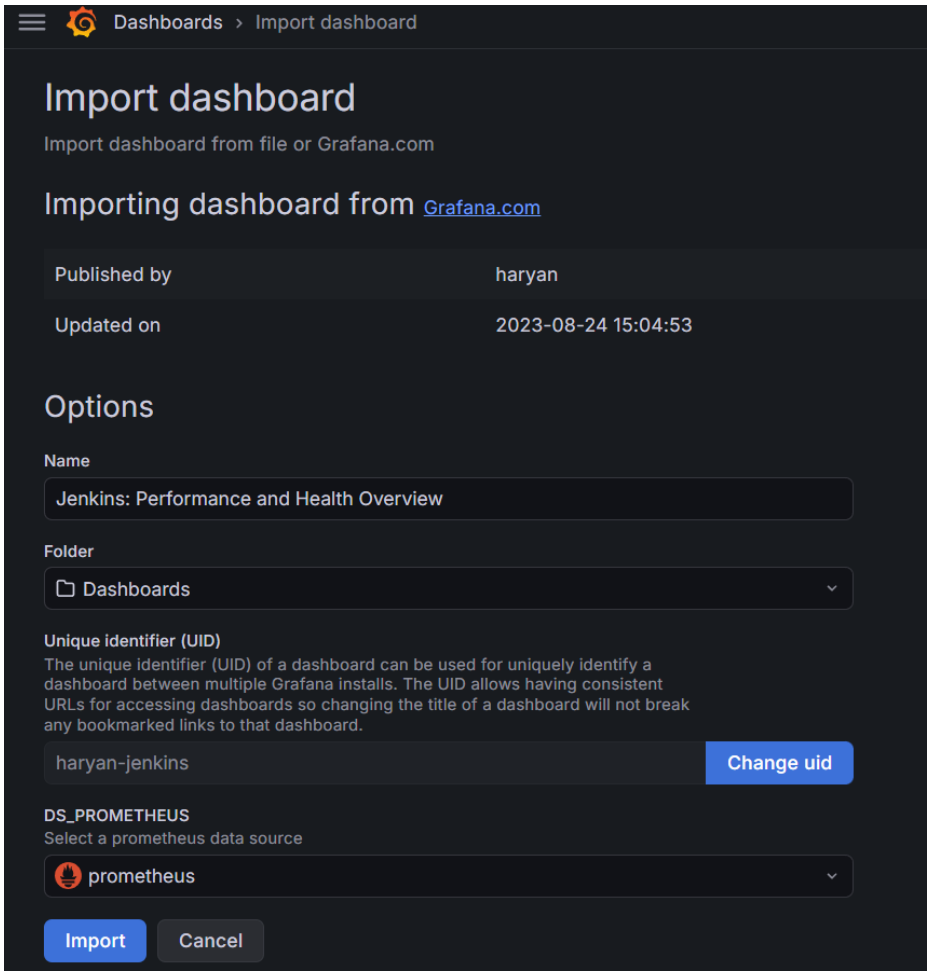


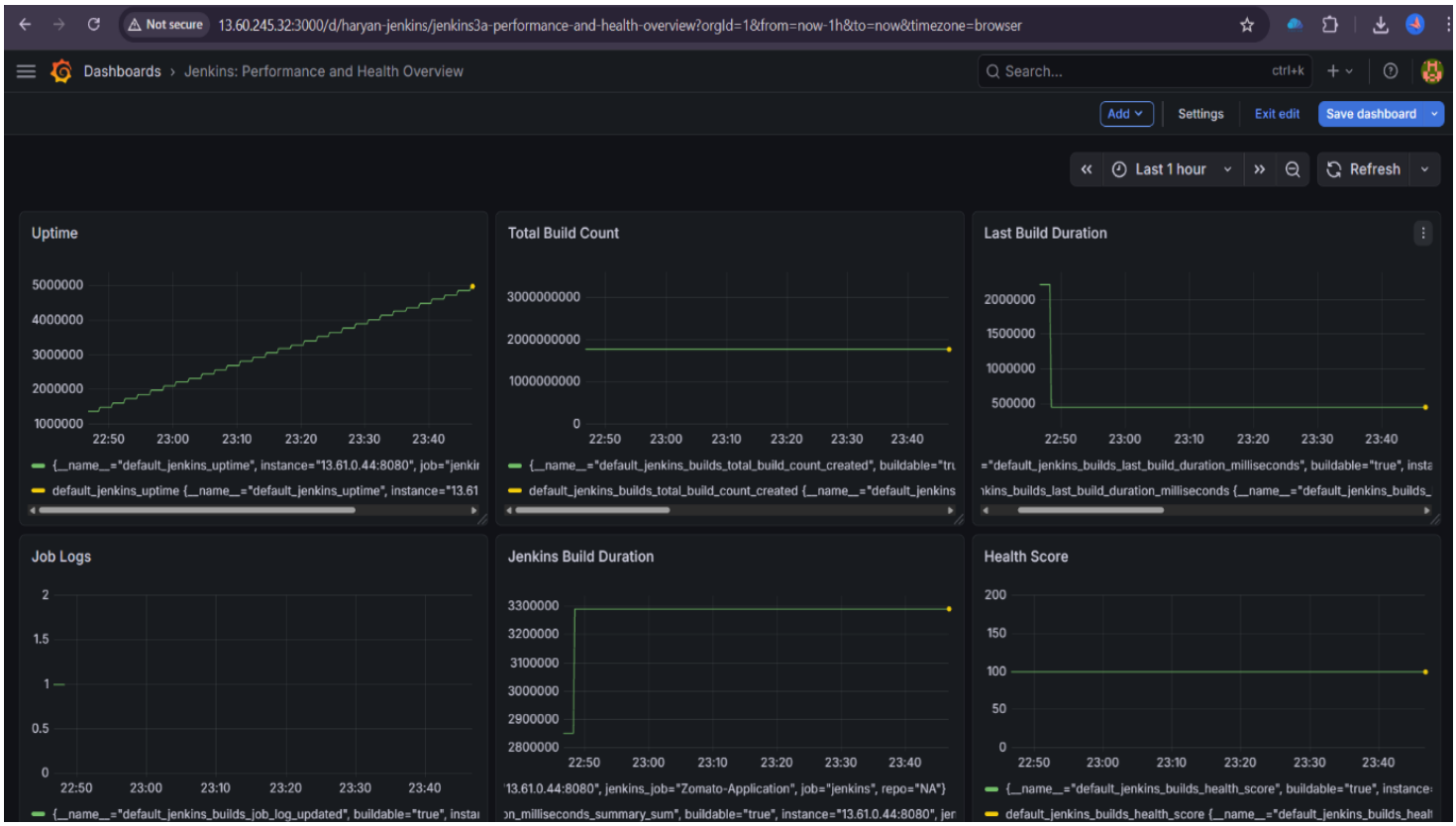

Dashboards › Import dashboard
Search...
Import dashboard
Import dashboard from file or Grafana.com
Importing dashboard from Grafana.com
Published by
rfmoz
Updated on
2025-09-27 22:45:03
Options
Name
Node Exporter Full
Folder
Dashboards
Unique identifier (UID)
The unique identifier (UID) of a dashboard can be used for uniquely identify a dashboard between multiple Grafana installs. The UID allows having consistent URLs for accessing dashboards so changing the title of a dashboard will not break any bookmarked links to that dashboard.
rYdddlPWk
Change uid
Import
Cancel


Dashboards › Node Exporter Full
Search...
ctrl+k
Edit
Export
Share
Datasource
prometheus
Job
node_exporter
Nodename
ip-10-0-12-135
Last 30 minutes
Refresh
1m
Instance
16.16.213.59:9100
GitHub
Grafana
Quick CPU / Mem / Disk
Pres...
CPU
0.3%
Mem
0.0%
I/O
0.1%
CPU...
1.1%
Sys ...
0.0%
RA...
57.6%
SW...
N/A
Root...
11.8%
C...
2
R...
914 MiB
S...
0 B
R...
27 GiB
Uptime
1.0 hours
Basic CPU / Mem / Net / Disk
CPU Basic
100%
80%
60%
40%
20%
0%
21:50
21:55
22:00
22:05
22:10
22:15
Busy System
Busy User
Busy Iowait
Busy IRQs
Busy Other
Idle
Memory Basic
896 MiB
768 MiB
640 MiB
512 MiB
384 MiB
256 MiB
128 MiB
0 B
21:50
21:55
22:00
22:05
22:10
22:15
Total
Used
Cache + Buffer
Free
Swap used

- Imported the Jenkins dashboard using Grafana Dashboard ID 9964.

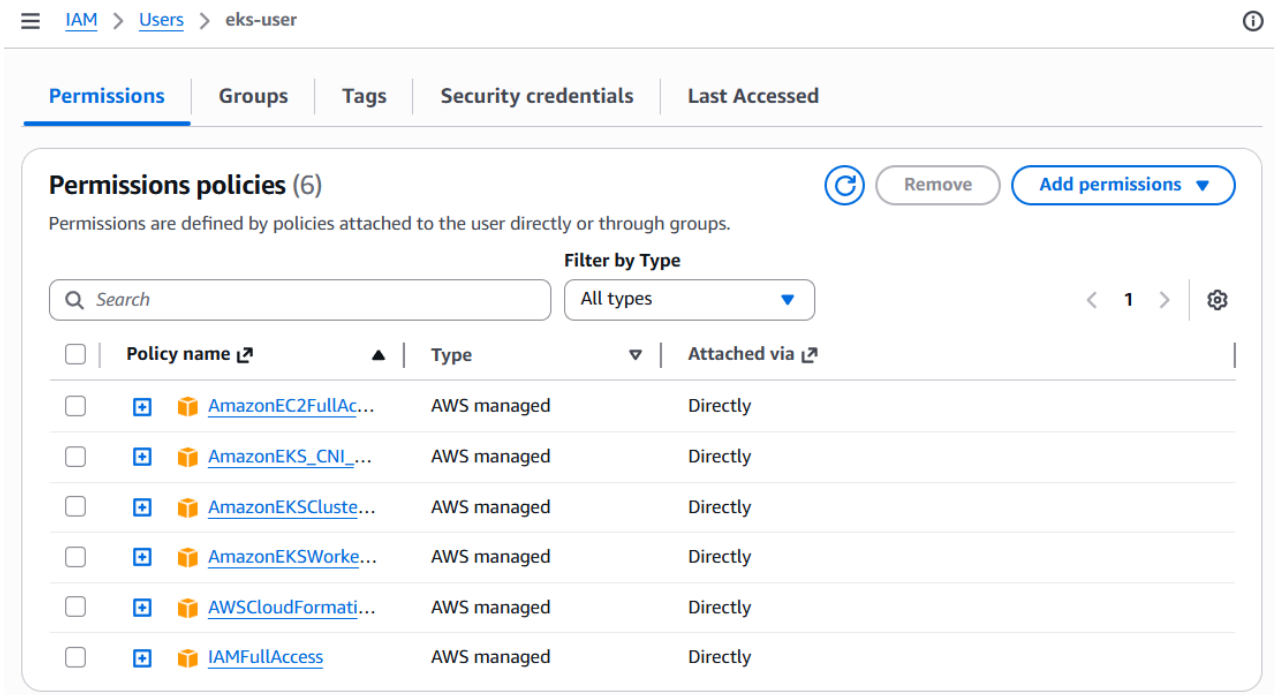


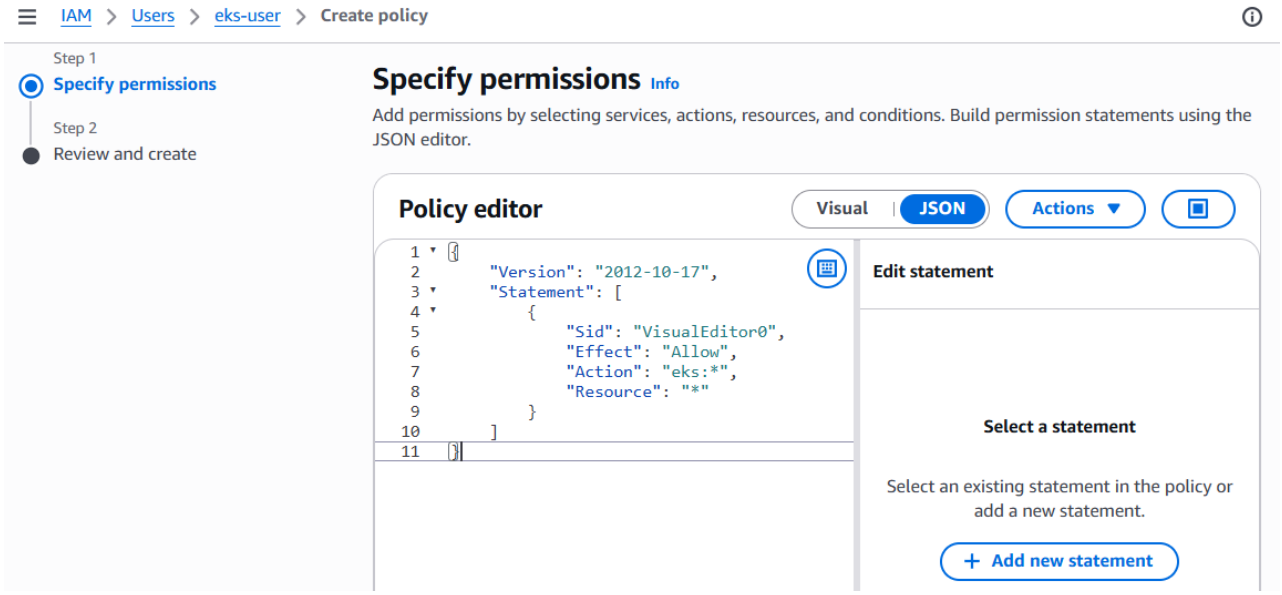


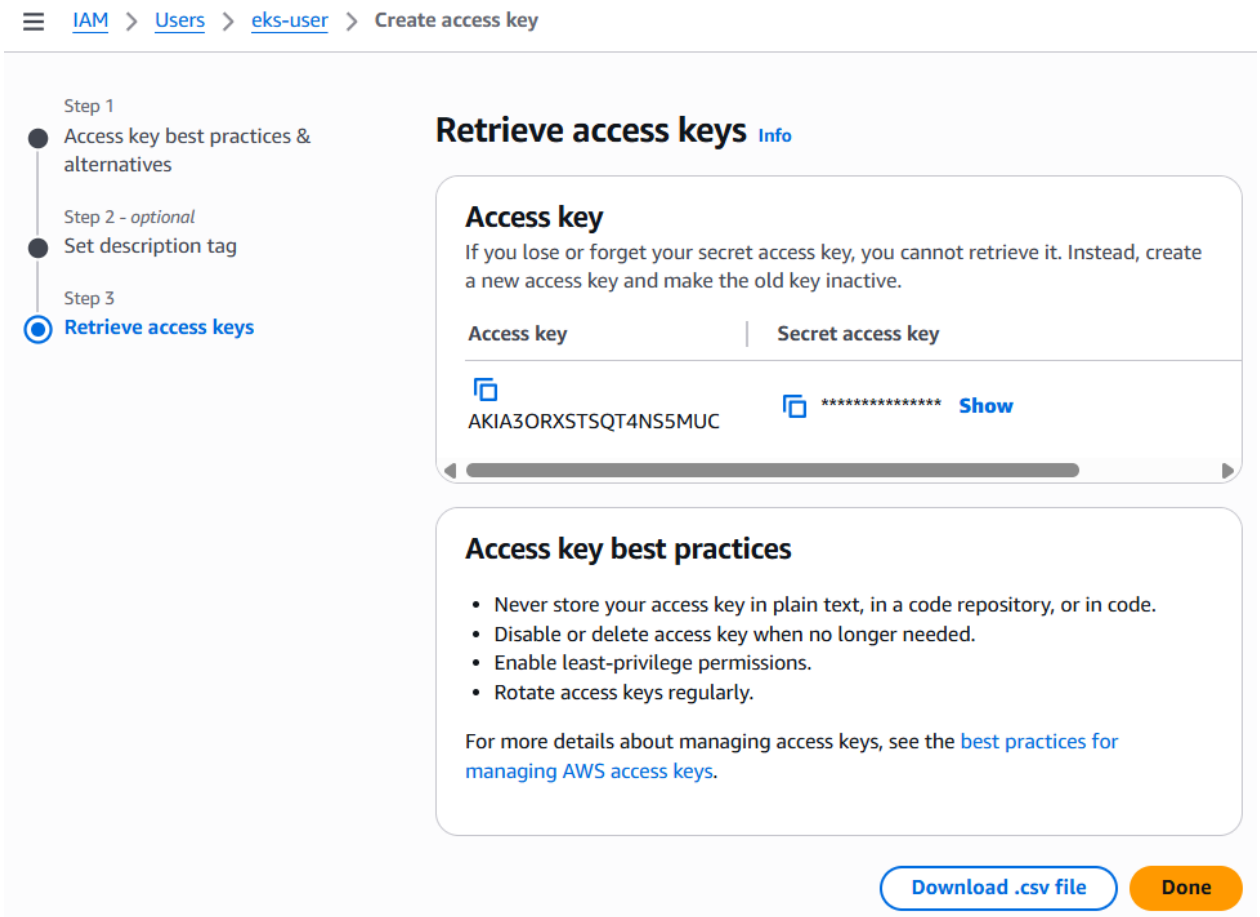


Phase 4: Kubernetes Deployment using Amazon EKS

- Created an IAM user named eks-user and attached necessary policies to manage the EKS cluster.




IAM > Users > eks-user > Create policy
Step 1
Specify permissions
Step 2
Review and create
Specify permissions Info
Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.
Policy editor
Visual
JSON
Actions
{
"Version": "2012-10-17",
"Statement": [
{
"Sid": "VisualEditor0",
"Effect": "Allow",
"Action": "eks:*",
"Resource": "*"
}
]
}
Edit statement
Select a statement
Select an existing statement in the policy or add a new statement.
+ Add new statement


IAM > Users > eks-user > Create access key
Step 1
Access key best practices & alternatives
Step 2 - optional
Set description tag
Step 3
Retrieve access keys
Retrieve access keys Info
Access key
If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.
Access key
Secret access key
AKIA3ORXSTSQT4NS5MUC
********** Show
Access key best practices
Never store your access key in plain text, in a code repository, or in code.
Disable or delete access key when no longer needed.
Enable least-privilege permissions.
Rotate access keys regularly.
For more details about managing access keys, see the best practices for managing AWS access keys.
Download .csv file
Done

- Installed AWS CLI on the BMS-Server and configured it using the IAM user's access key and secret key.

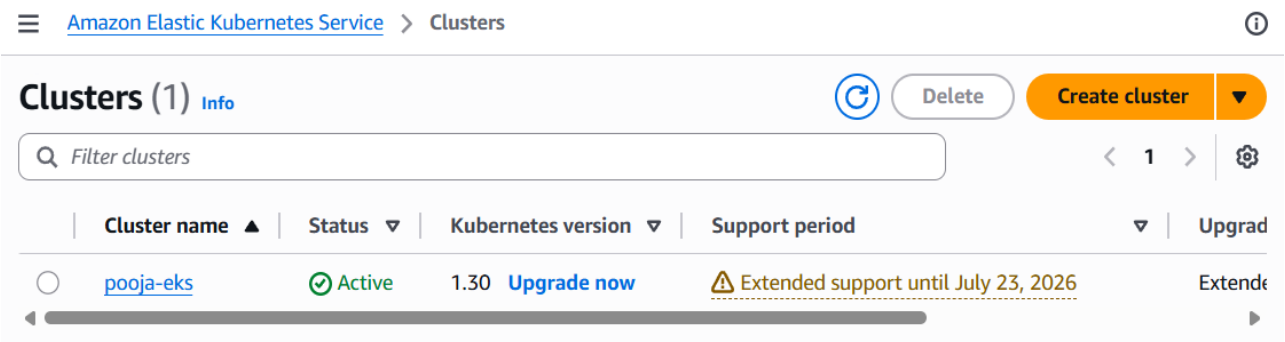
```
You can now run: /usr/local/bin/aws --version
AWS Access Key ID [None]: AKIA3ORXSTSQT4NS5MUC
AWS Secret Access Key [None]: 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
Default region name [None]: eu-north-1
Default output format [None]: json
root@ip-10-0-6-33:/home/ubuntu# aws --version
aws-cli/2.34.0 Python/3.13.11 Linux/6.14.0-1018-aws exe/x86_64.ubuntu.24
root@ip-10-0-6-33:/home/ubuntu#
```

- Installed kubectl and eksctl to create and manage the Kubernetes cluster.

```
2. 13.49.18.254 (ubuntu)
root@ip-10-0-6-33:/home/ubuntu# kubectl version --client
Client Version: v1.35.2
Kustomize Version: v5.7.1
root@ip-10-0-6-33:/home/ubuntu# eksctl version
0.223.0
root@ip-10-0-6-33:/home/ubuntu#
```

- Created an EKS cluster named **pooja-eks** and verified its status in the AWS Console.

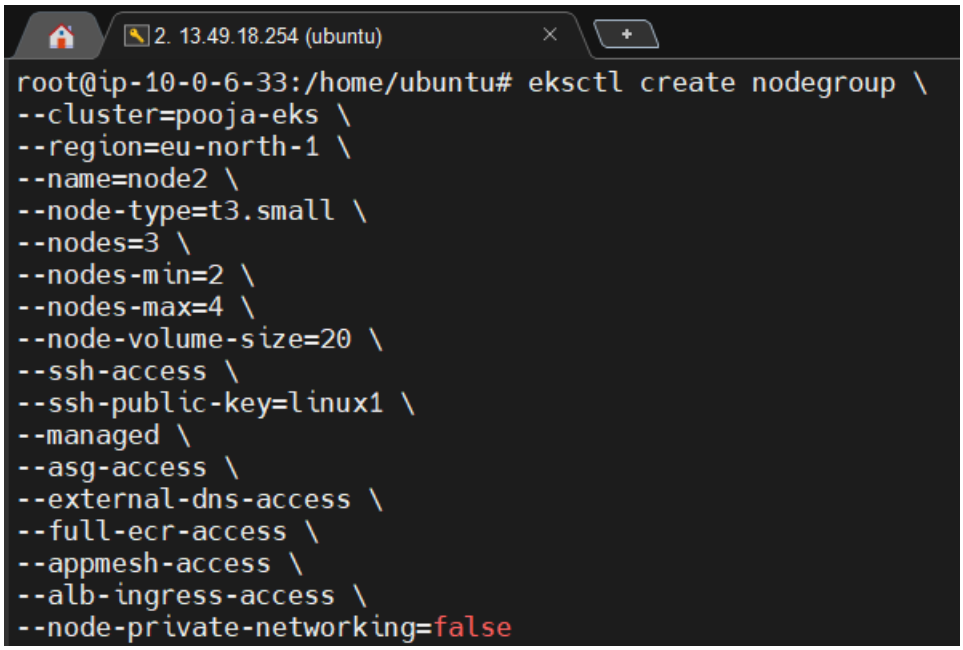
```
2. 13.49.18.254 (ubuntu)
root@ip-10-0-6-33:/home/ubuntu# eksctl create cluster \
--name=pooja-eks \
--region=eu-north-1 \
--version=1.30 \
--vpc-public-subnets=subnet-02c17cda1d166a85b,subnet-061485a8046389ab0 \
--without-nodegroup
```



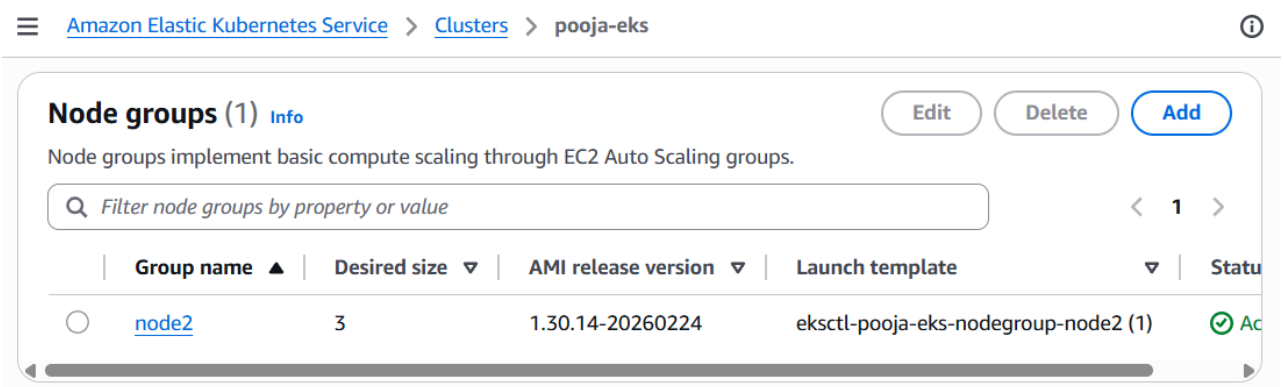
- Associated an OIDC identity provider to enable IAM roles for service accounts.

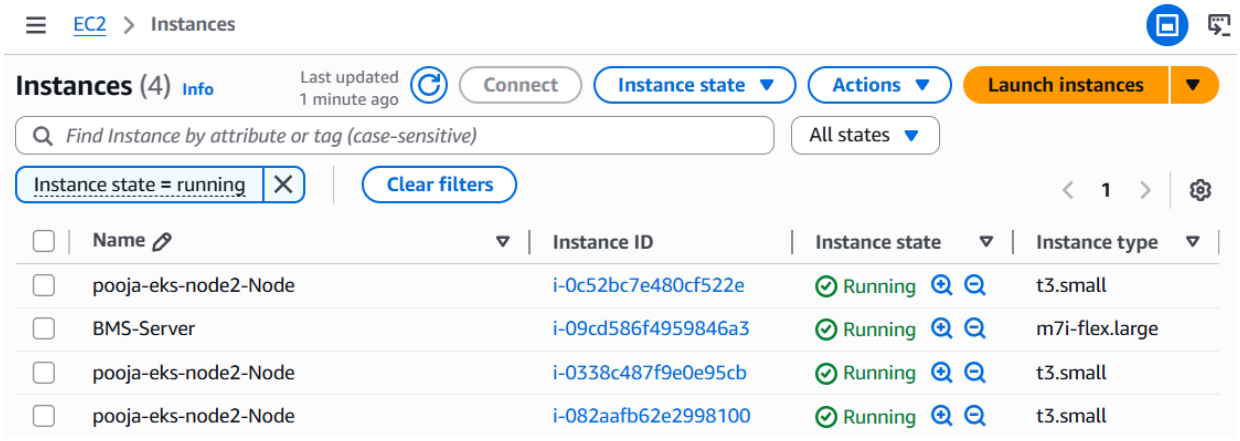
```
root@ip-10-0-6-33:/home/ubuntu# eksctl utils associate-iam-oidc-provider \
> --region eu-north-1 \
> --cluster pooja-eks \
> --approve
2026-02-28 11:01:30 [ℹ]  will create IAM Open ID Connect provider for cluster "pooja-eks" in "eu-north-1"
2026-02-28 11:01:31 [✔]  created IAM Open ID Connect provider for cluster "pooja-eks" in "eu-north-1"
root@ip-10-0-6-33:/home/ubuntu# npm --version
9.2.0
root@ip-10-0-6-33:/home/ubuntu#
```

- Created a managed Node Group in public subnets with required add-ons.



- Verified worker nodes were successfully registered.





- Created Kubernetes YAML files for Deployment and Service.

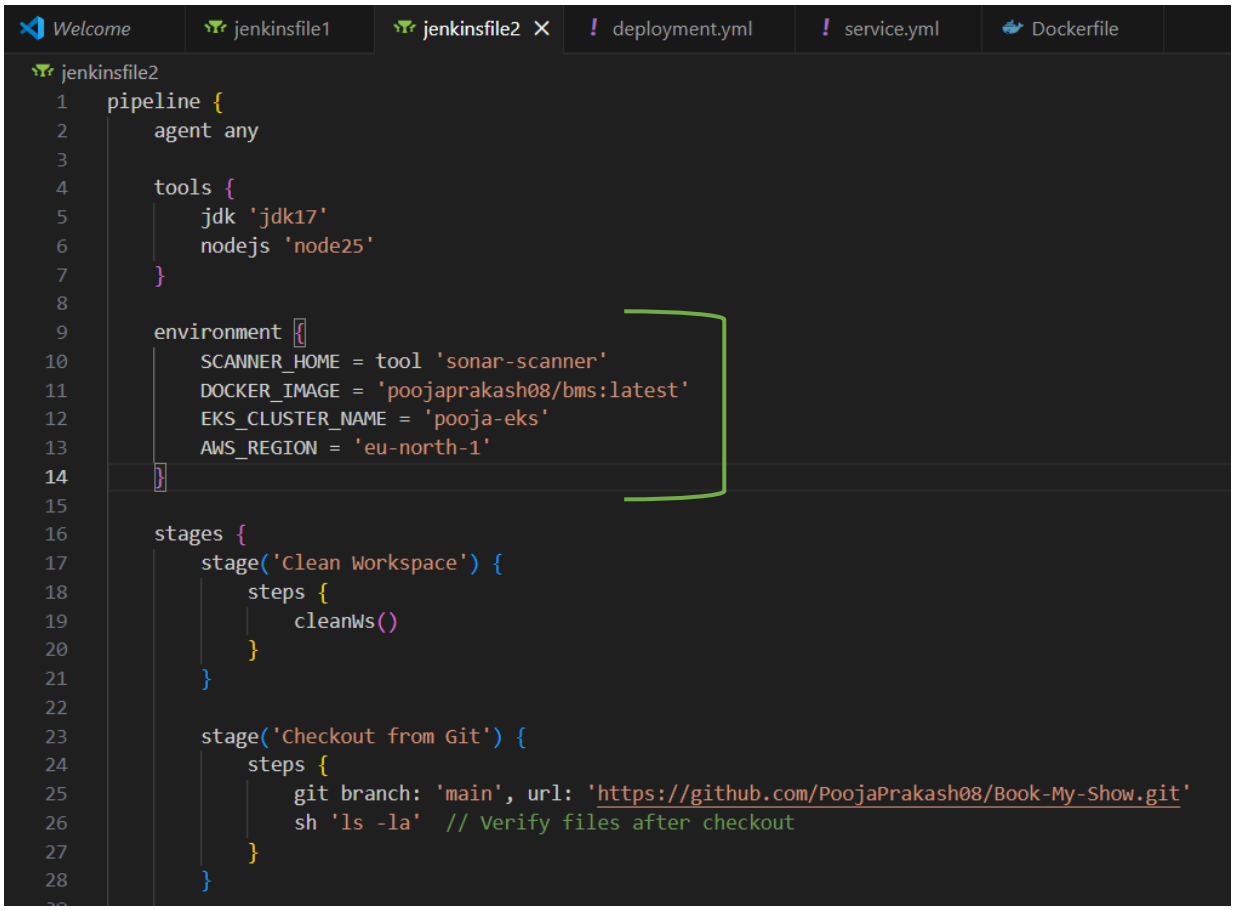
service.yml

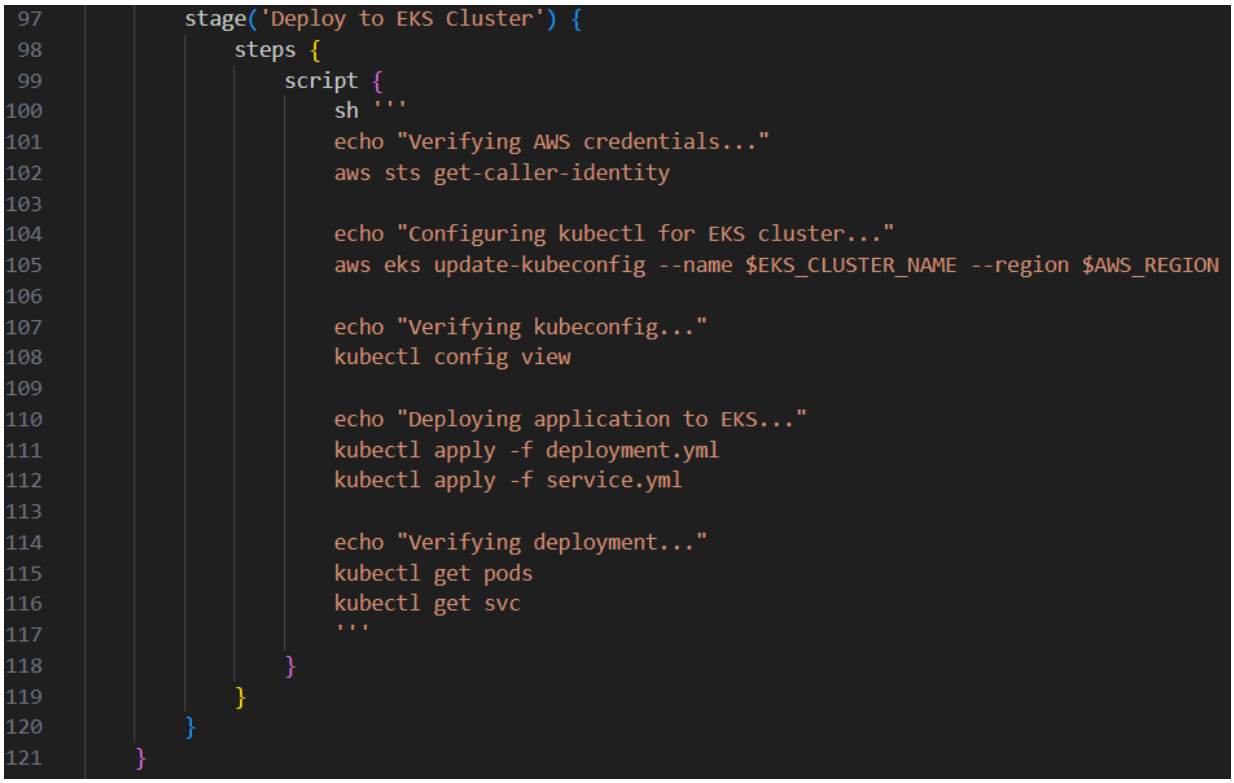
```yaml
apiVersion: v1
kind: Service
metadata:
  name: bms-service
  labels:
    app: bms
spec:
  type: LoadBalancer
  ports:
  - port: 80
    targetPort: 3000
  selector:
    app: bms
```

deployment.yml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: bms-app
  labels:
    app: bms
spec:
  replicas: 2
  selector:
    matchLabels:
      app: bms
  template:
    metadata:
      labels:
        app: bms
    spec:
      containers:
      - name: bms-container
        image: poojaprakash08/bms:latest
        ports:
        - containerPort: 3000
```

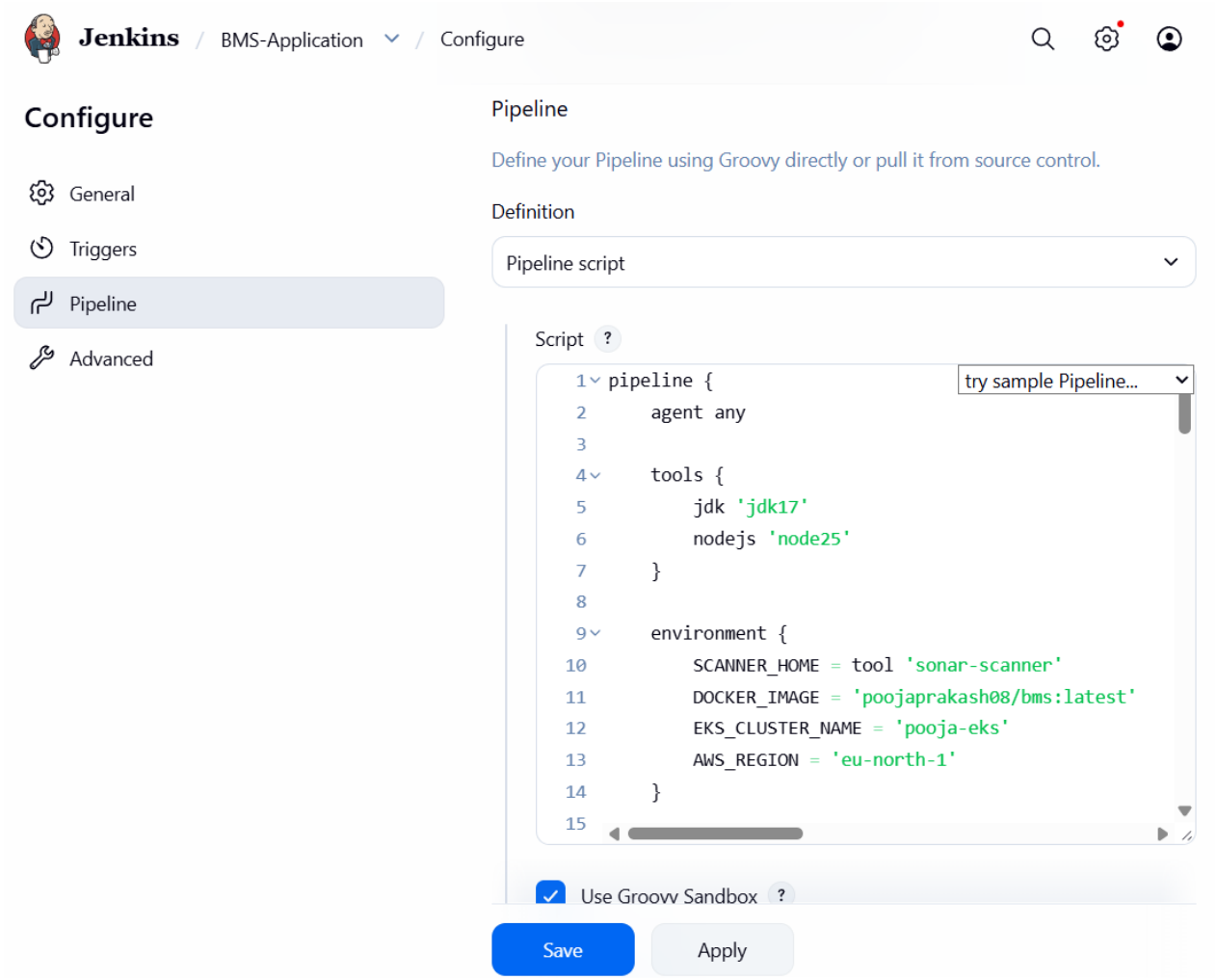
- Updated the Jenkinsfile to deploy the application into the Kubernetes cluster.

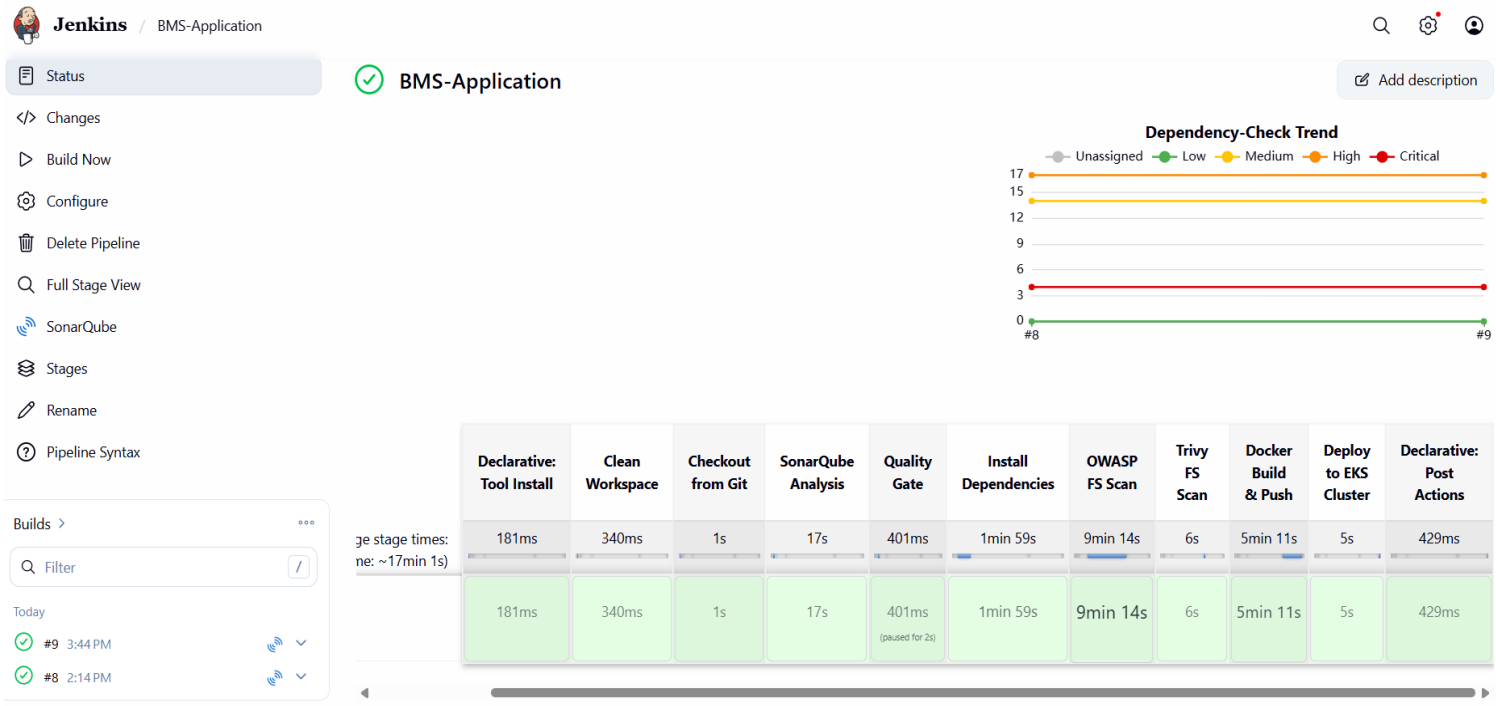




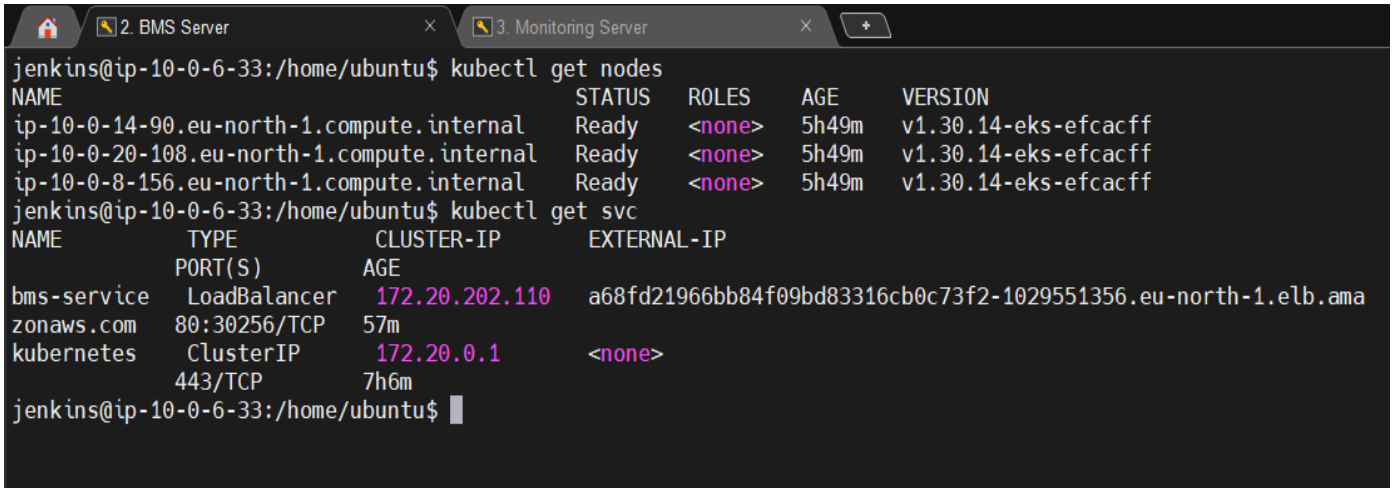
```
jenkins@ip-10-0-6-33:/home/ubuntu$ aws eks update-kubeconfig --name pooja-eks --region eu-north-1
Added new context arn:aws:eks:eu-north-1:787169320097:cluster/pooja-eks to /var/lib/jenkins/.kube/config
jenkins@ip-10-0-6-33:/home/ubuntu$
```

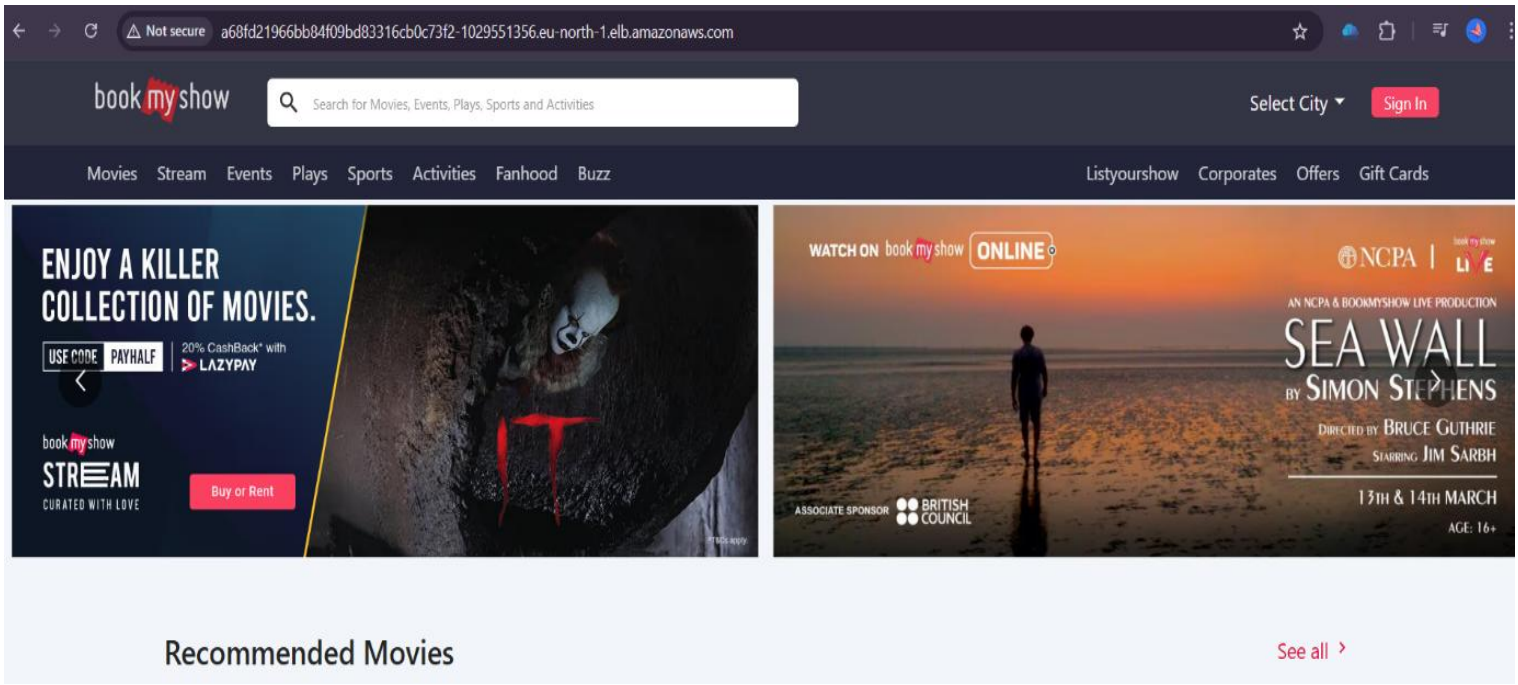
- Updated the pipeline job with the modified Jenkinsfile and rebuilt the pipeline.





- Successfully accessed the Book My Show application using the Load Balancer URL provisioned by Kubernetes.





- ✓ This project successfully demonstrates a complete DevOps lifecycle implementation for the Book My Show application, including CI/CD automation, containerization, security scanning, monitoring and Kubernetes-based deployment on AWS. The integration of Jenkins, Docker, SonarQube, OWASP, Trivy, Prometheus, Grafana and Amazon EKS ensures a scalable, secure and production-ready deployment architecture.